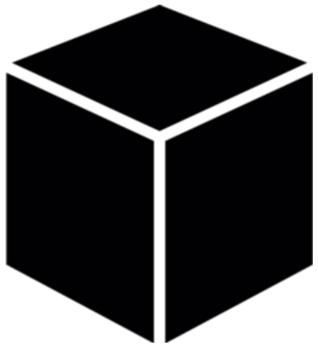
Constraint Programming



Search

Module Overview

- Motivation
- Heuristics
  - Value selection heuristic
  - Variable selection heuristics
    - First Fail
    - Degree
    - Impact
    - Activity
    - Most Recent Conflict
- Strategies (a.k.a. Meta-heuristics)
  - Discrepancy search
  - LNS & restarts

Motivation

Refresher

Black-Box Search: The Vision

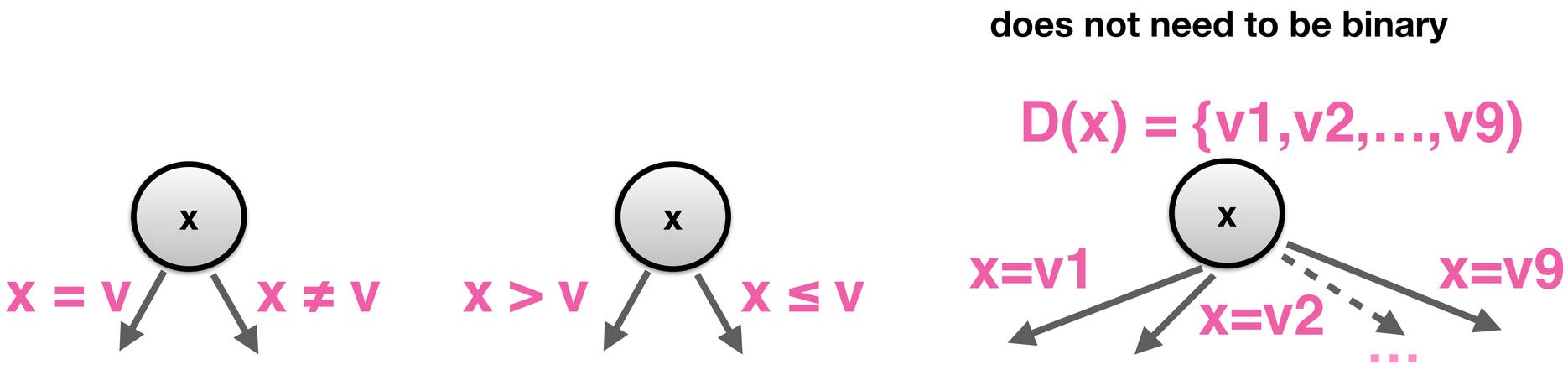
- Constraint Program = Model + Search

The vision is that we should
not have to worry too much about the search.
A CP solver should be smart enough to learn
or select a good search for the model.

- Since ~2004, many research efforts have been devoted to making this vision true.

Refresher: Branching = 2-Step Choice

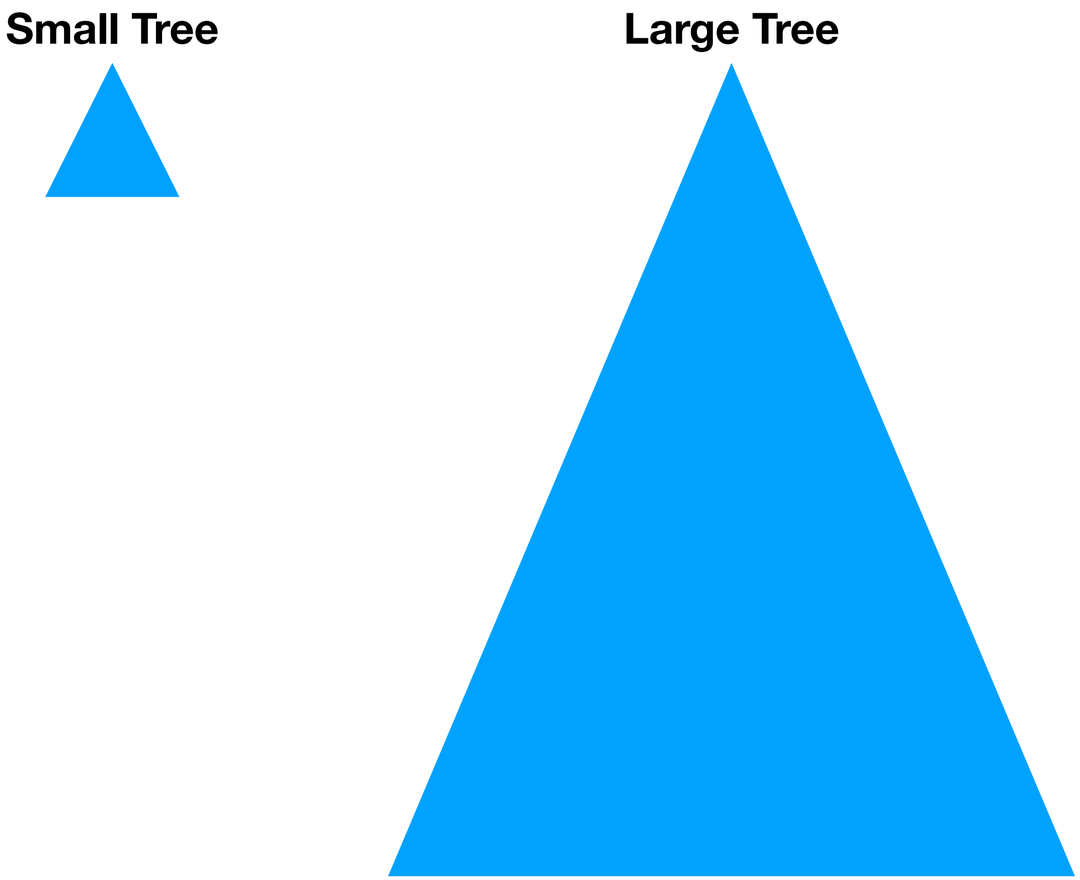
- Variable selection.
- Value selection & partition method.



The branching decisions can have a strong impact on the size of the search tree.

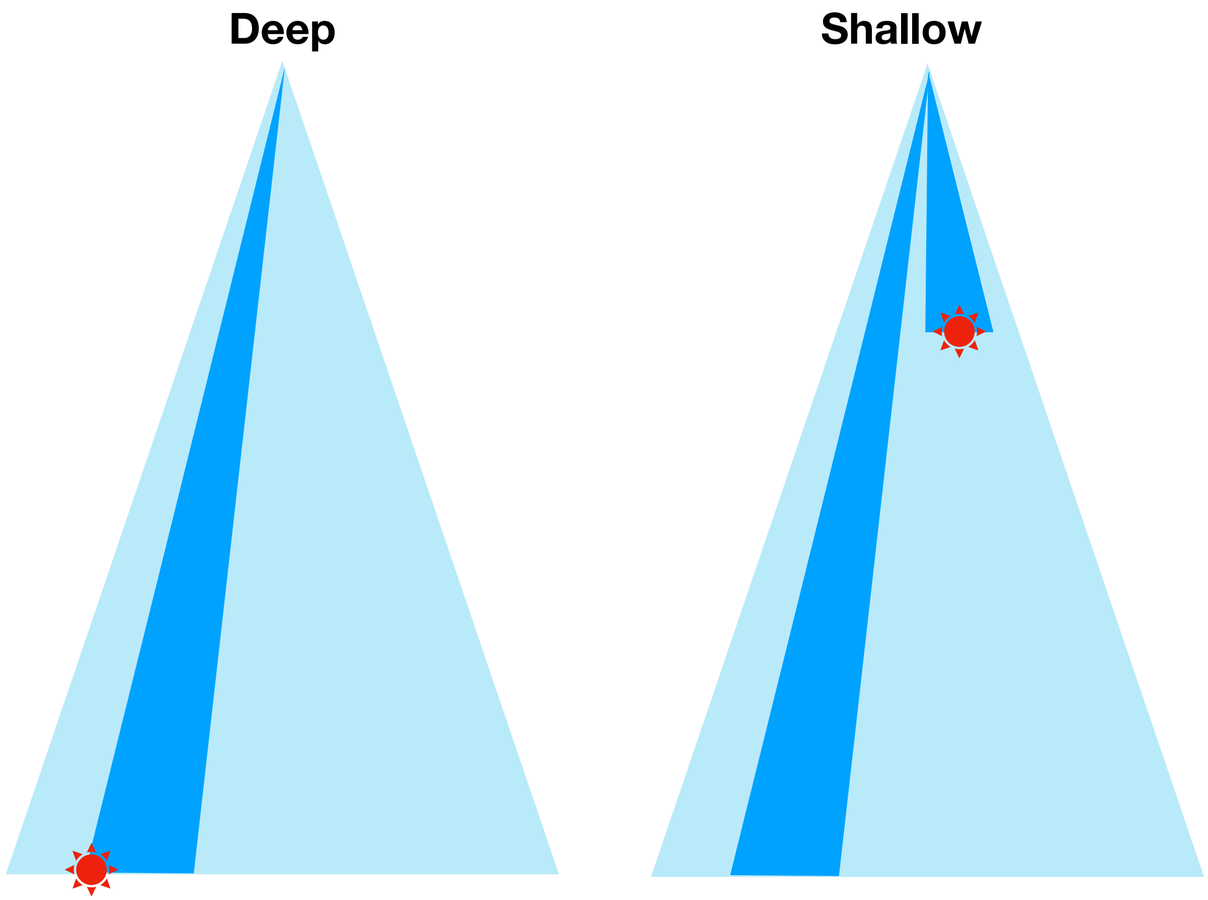
Searching…

- What is the big deal ?
- It is all about the tree!
  - Size/Shape
  - Depth of solutions
  - Location of solutions



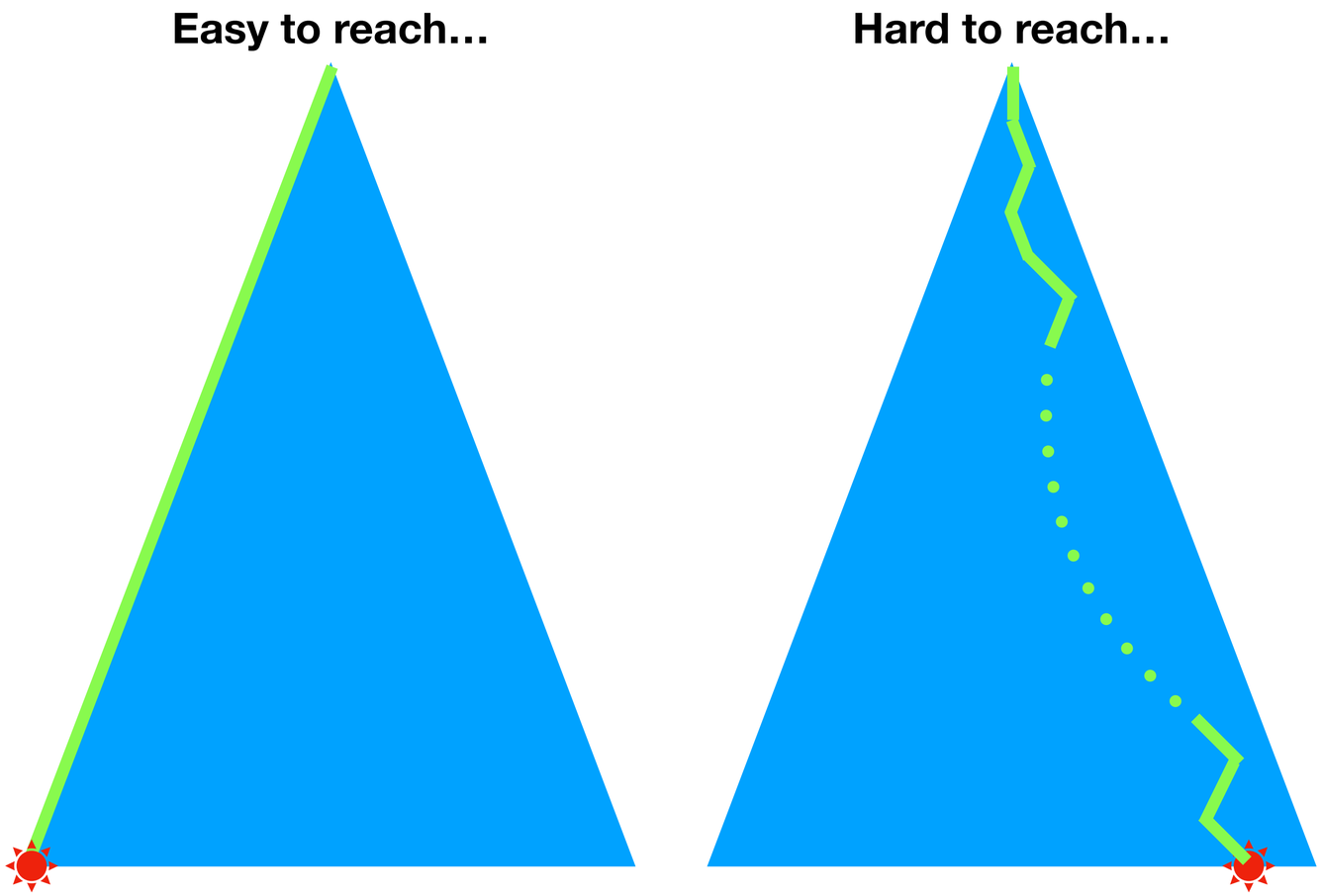
Searching…

- What is the big deal ?
- It is all about the tree!
  - Size/Shape
  - Depth of solutions
  - Location of solutions



Searching…

- What is the big deal ?
- It is all about the tree!
  - Size/Shape
  - Depth of solutions
  - Location of solutions



Bottom line

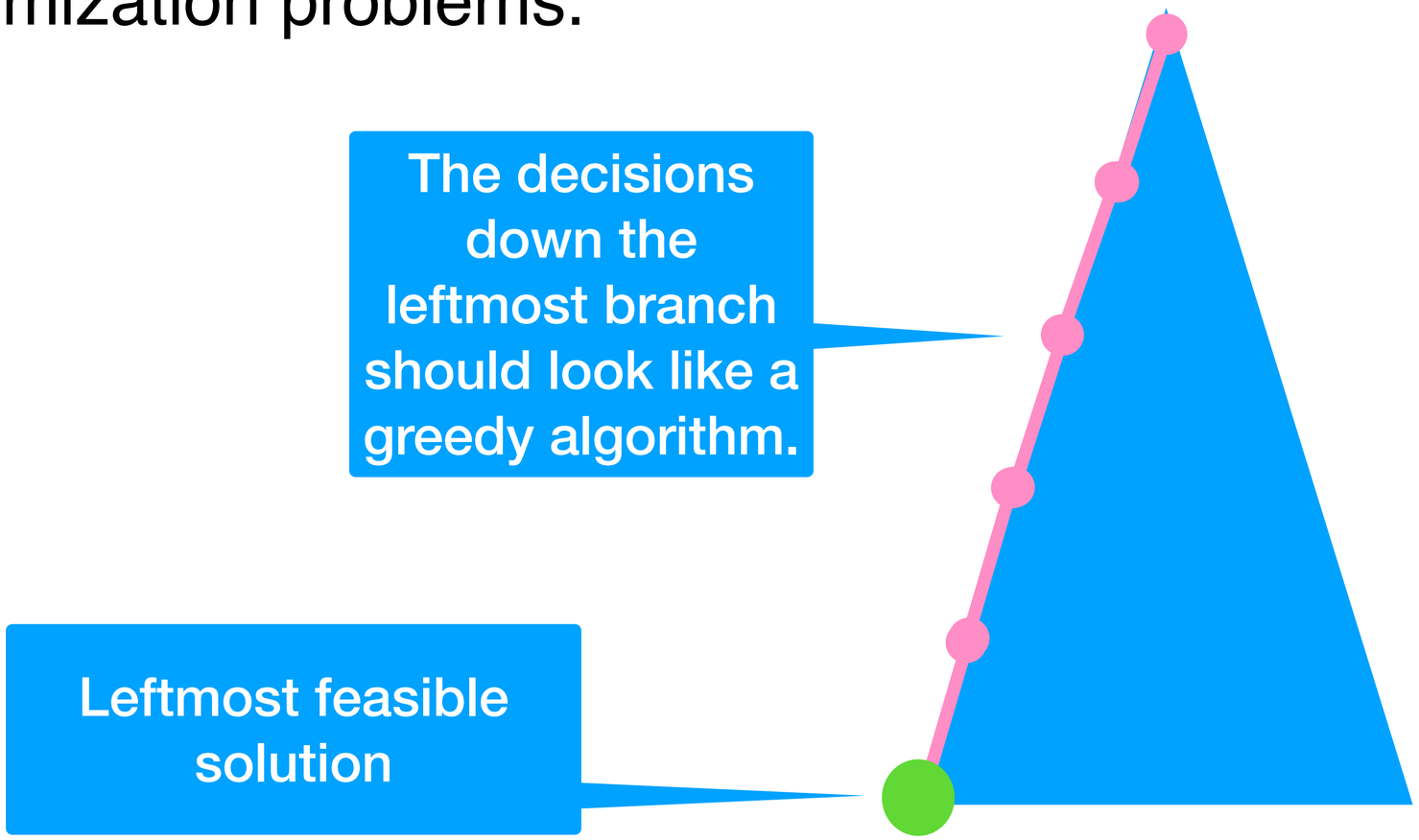
- Variable selection has an impact
  - On Size of tree (because of propagation)
  - On quality of solutions for COP
  - On depth where (good) solutions are
- Value selection has an impact
  - On size of tree
  - On location of solutions (heuristics recommendation are “far left”)
  - On quality of solutions for COP
- Strategies have an impact
  - On how quickly you hit a solution that is good or hard to reach

Value Heuristic

For optimization problems

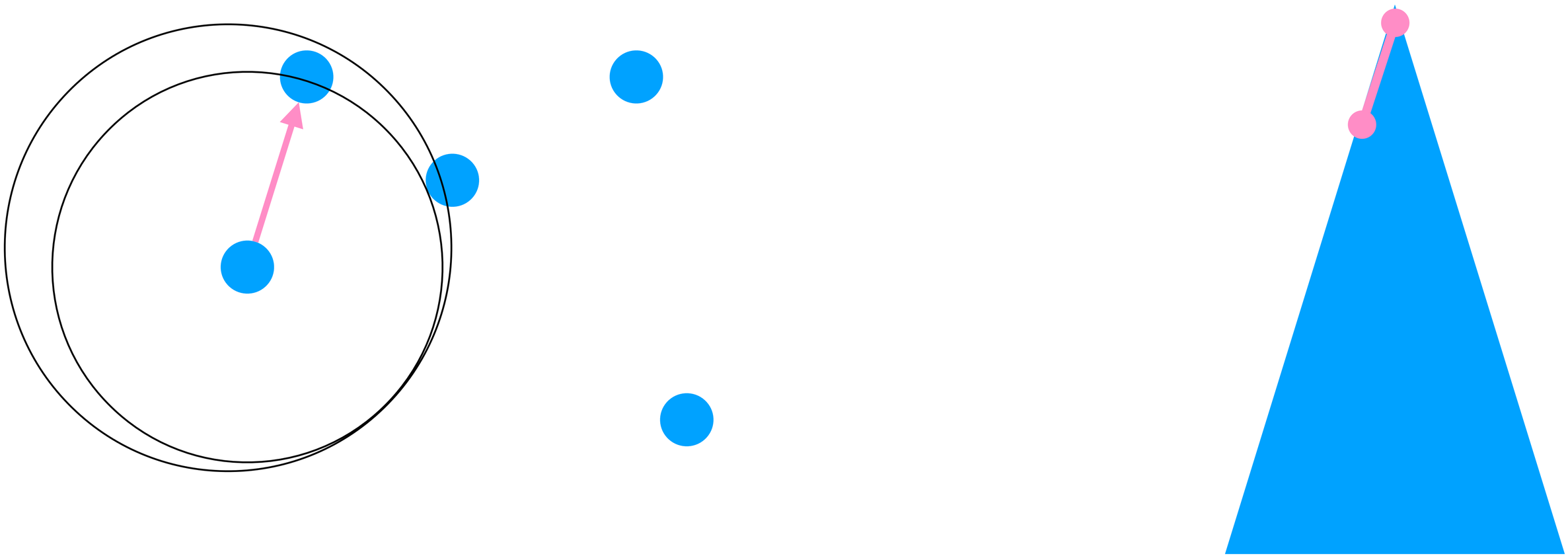
Value Selection for Optimization Problems

- The leftmost feasible solution is very important for pruning during branch-and-bound search.
- Solution quality is mostly impacted by the *value* selection heuristic for optimization problems.

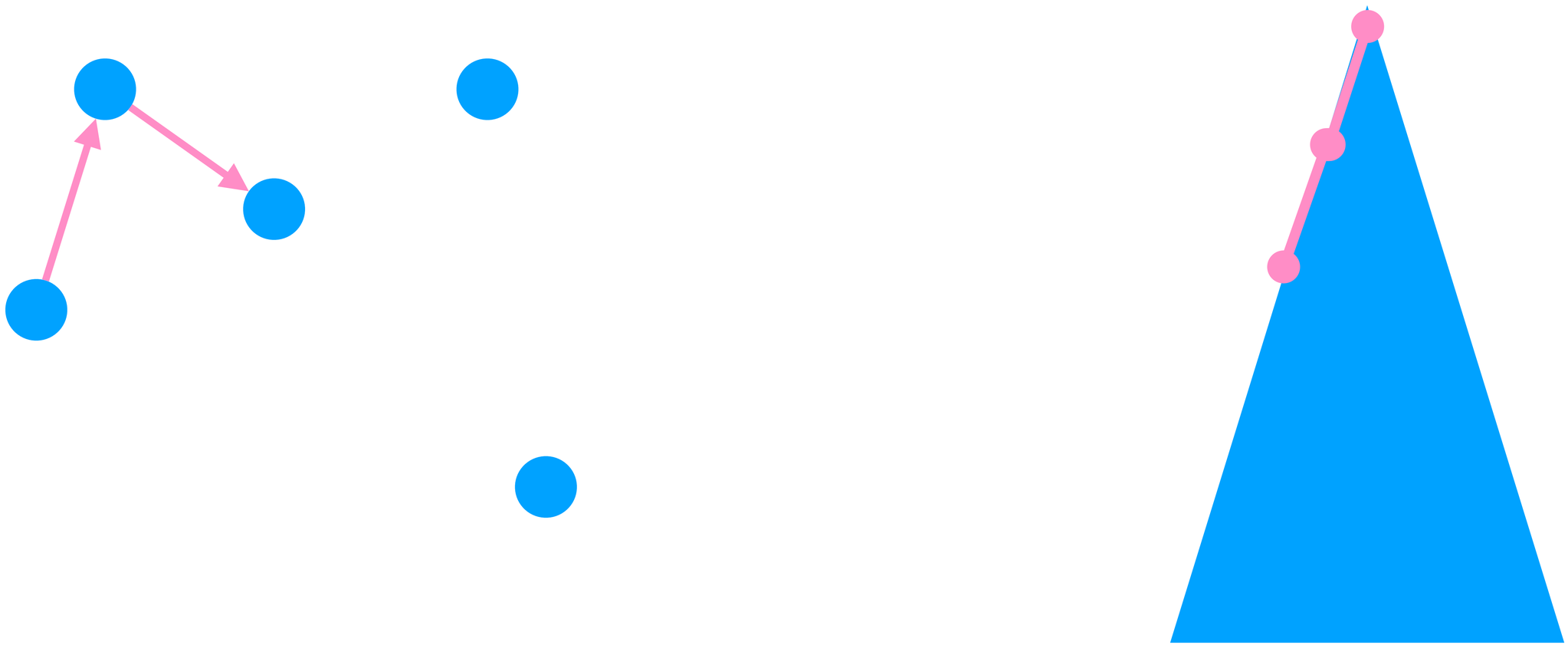


Example: Traveling Salesperson Problem

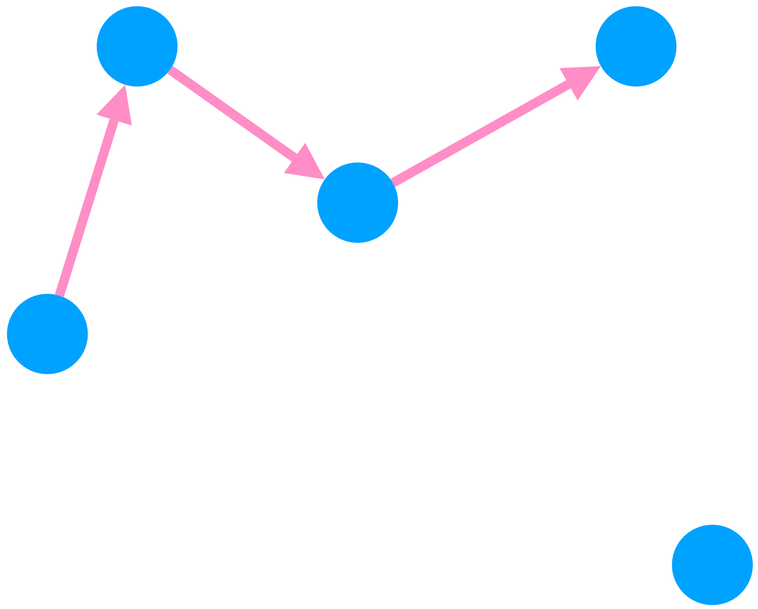
Greedily, we would adopt a nearest-neighbor value selection heuristic:

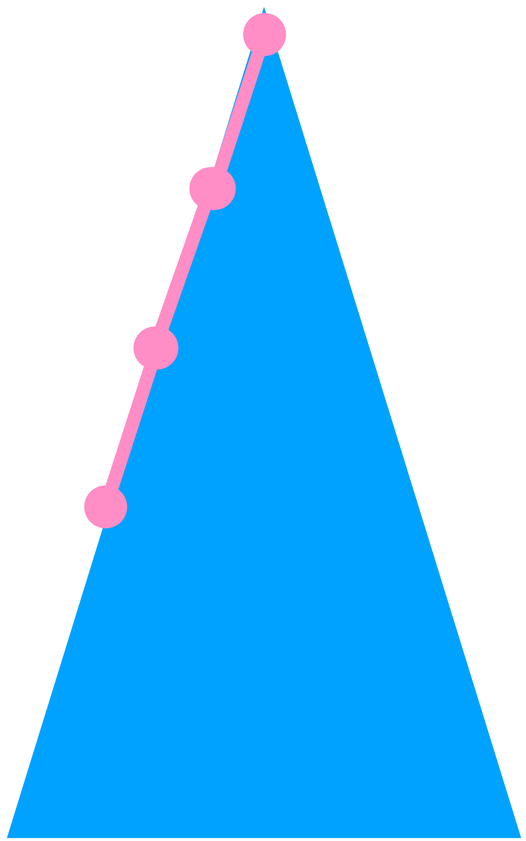


Example: Traveling Salesperson Problem

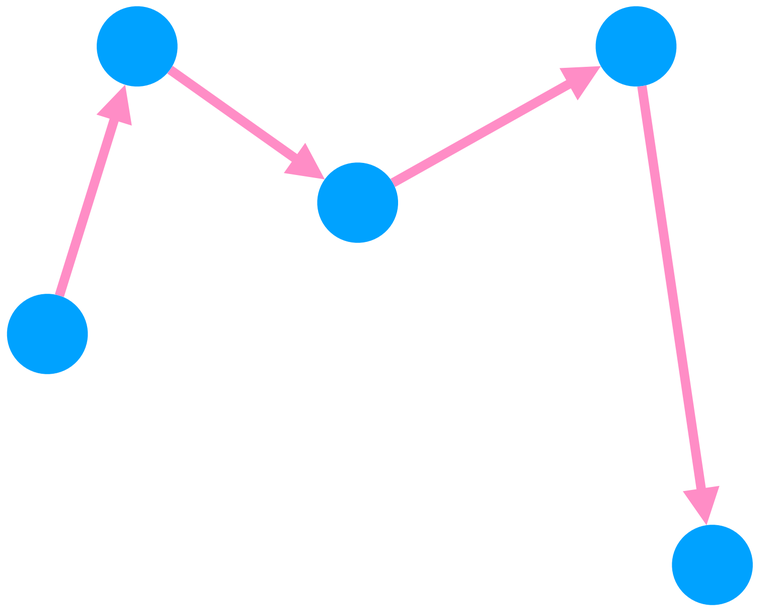


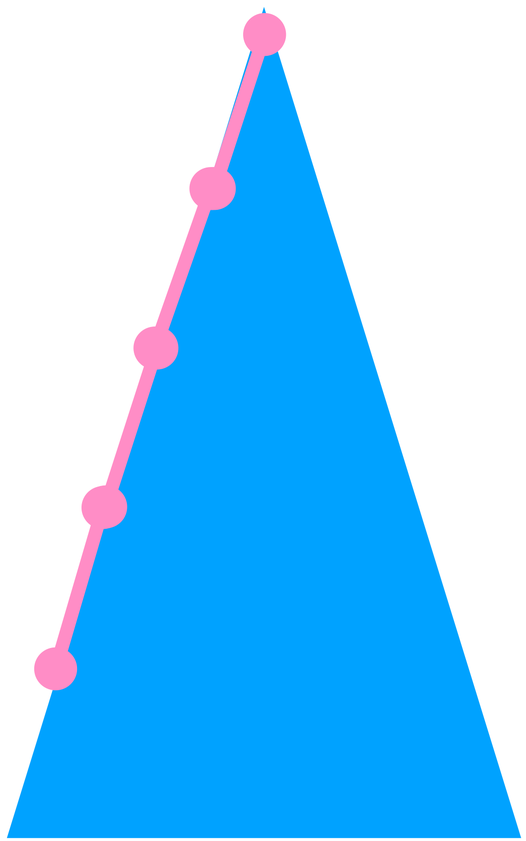
Example: Traveling Salesperson Problem





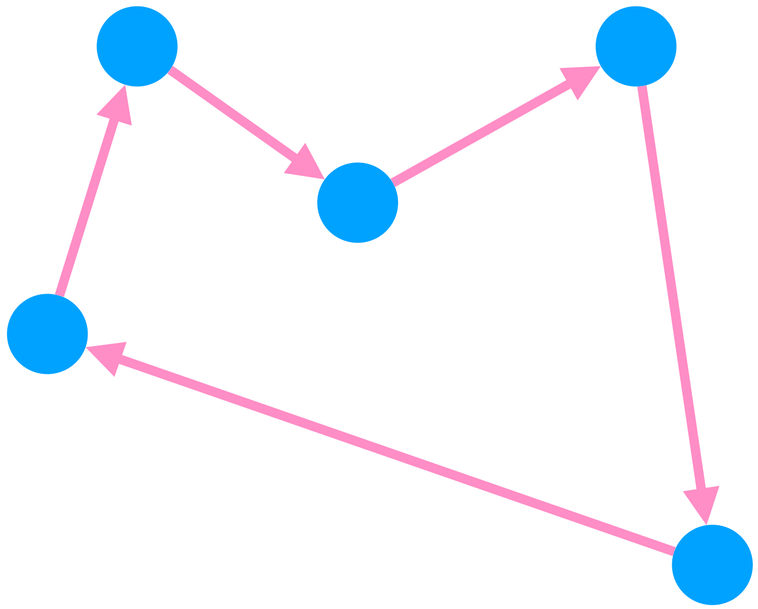
Example: Traveling Salesperson Problem

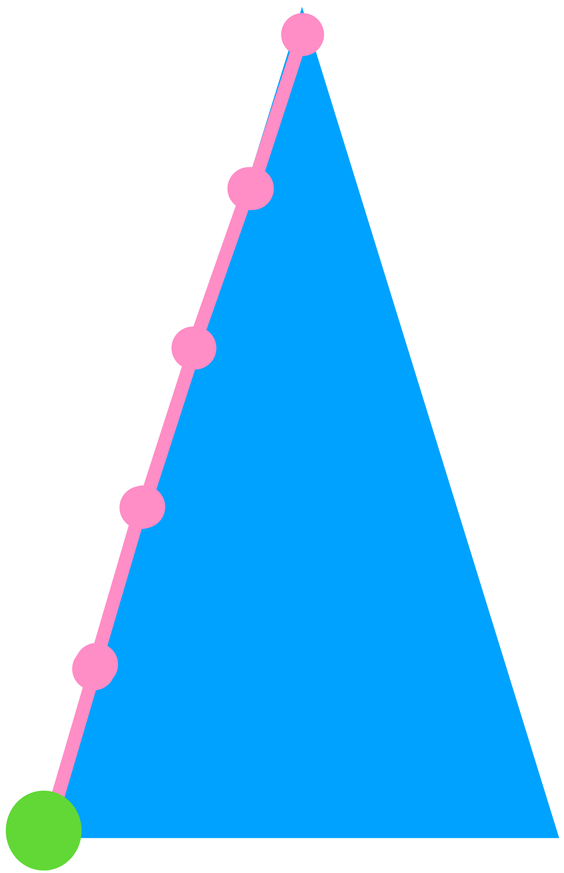




Example: Traveling Salesperson Problem

Can we automate this for *any* problem in order to find a good first feasible solution?





Making the First Solution Good!

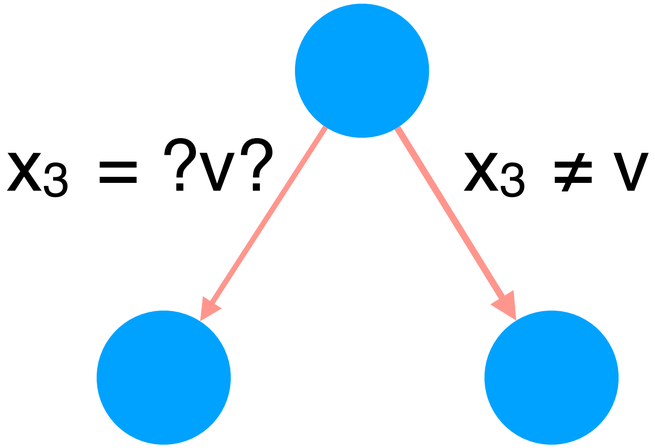
Branching = 2 steps:

1. Variable selection.
2. Value selection & partition selection: bound-impact value selector (BIVS) selects a value with the smallest objective lower bound after propagation of the fixing of the selected variable to this value (when minimizing).

Jean-Guillaume Fages and Charles Prud'homme.
Making the first solution good!
International Conference on Tools with Artificial Intelligence (*ICTAI*), 2017.

Making the First Solution Good!

- Minimize $O = f(x_1,x_2,\ldots,x_{10})$ subject to some constraints.
- The variable selection heuristic selects x_3.
- The current domain is $D(x_3) = \{v_1,v_2,v_3,v_4\}$.




```
best = null
bestObj = +inf
for v in {v1,v2,v3,v4}
    sm.saveState()
    cp.post(x3 = v)
    if min(O) < bestObj:
        best = v
        bestObj = min(O)
    sm.restoreState()
branch {cp.post(x3 = best)}
       {cp.post(x3 != best)}
```

Making the First Solution Good: Advice

- This procedure (for making the first solution good) is quite costly: fixpoint computation for *every* value + save & restore the state.
- The advice is to do this only for discovering the *first* feasible solution and then to use a more naïve (but faster) value selection heuristic.

Value Selection: Phase Saving

- If a value v was successfully used for fixing a selected variable x
  - that is: if branching on x=v did not lead to failure of the fixpoint algorithm

  then store the value v as the last success value of x.
- Each time the variable x is selected by the variable selection heuristic, first try its last success value.

```
SearchStatistics stats = makeDfs(cp,
        selectMin(q,
                qi -> qi.getSize() > 1,
                qi -> qi.getSize(),
                qi -> {
                    int v = qi.getMin();
                    return branch(() -> equal(qi,v),
                                  () -> notEqual(qi,v));
                }
        )
```

One can easily implement a default search doing phase saving.

Variable Selection

- Introduction to First-Fail (FF) Principle
- A first instantiation of the FF based on dom size

Branching

Reminder:

- Variable selection.
- Value selection & partition selection.

For the rest of this lecture, we focus on *variable* selection.

First-Fail Principle

First-fail for variable selection:

Since all variables must eventually be fixed,
if there are no solutions under a node (failure), then we prefer to detect this as soon as possible, so that not too much time is spent exploring the subtree under the node.

"To succeed, try first where you are most likely to fail."

Robert M. Haralick and Gordon L. Elliott.
Increasing tree search efficiency for constraint satisfaction problems.
International Joint Conference on Artificial Intelligence (*IJCAI*), 1979.

First-Fail Principle for Variable Selection

- Can be implemented in various ways:
  - Min-Dom
  - Dom+Deg
  - Dom/Deg
  - Dom/Wdeg
  - Impact-based search
  - Activity-based search
  - Last-conflict search
  - Conflict-ordering search
  - …

Min-Dom Heuristic

- Min-Dom: Select an unfixed variable with the smallest domain size.
- This heuristic was shown experimentally to minimize search-tree depth.
- It is quite intuitive that for D(x1) = {1,2} and D(x2) = {1,…,100} branching on x1 first is likely to trigger more propagation.

Min-Dom Heuristic

```
int n = 8;
Solver cp = Factory.makeSolver(false);
IntVar[] q = Factory.makeIntVarArray(cp, n, n);
// constraints …

DFSearch search = Factory.makeDfs(cp, () -> {
    IntVar qs = selectMin(q,
            qi -> qi.size() > 1,
            qi -> qi.size());
    if (qs == null) return EMPTY;
    else {
        int v = qs.min();
        return branch(() -> Factory.equal(qs, v),
                () -> Factory.notEqual(qs, v));
    }
});

search.onSolution(() -> println("solution:" + Arrays.toString(q)));
SearchStatistics stats = search.solve();
```

Select an unfixed variable with the smallest domain size.

Variable Selection

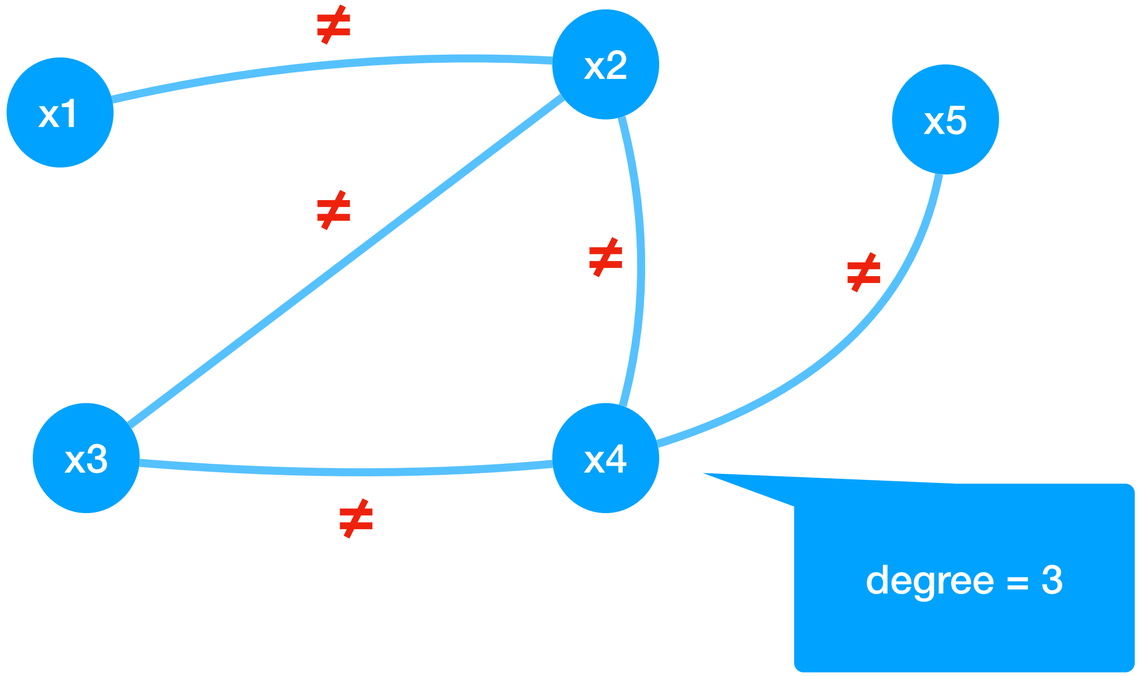
- First-Fail (FF) Principle based on degree in the constraints graph

Degree-Based Heuristics

If a ***variable is involved in many constraints***,
then it is likely that any filtering of the domain of this variable
will trigger some filtering for other variables of those constraints.

Dom+Deg and Dom/Deg

- Degree of a variable x = the number of constraints on x.
- Dom+Deg: Min-Dom, and break ties with the degree.
- Dom/Deg: Select an unfixed variable with the smallest ratio between domain size and degree.



Weighted Degree (Wdeg) Heuristic

- Idea: Introduce learning to find out what the “difficult” variables are.
- Each time a constraint fails, its weight is increased (by +1).
  - The idea is that if a constraint has failed a lot in the past, then it will probably continue this trend in the future.
- Weighted degree of a variable x = the sum of the weights of the constraints on x and at least one other unfixed variable.

- Dom/Wdeg: Select the first unfixed variable with the smallest ratio between domain size and weighted degree.

- Weakness: A constraint that fails may not be the only guilty one for the failure (cascade of propagations in the fixpoint algorithm).

Dom/Wdeg Implementation

- Not shipped with MiniCP, but quite easy to implement.
- Each constraint should be aware of the variables in its scope.
- Add a method called `scope` to the class `Constraint`:
  it returns the list of variables in the scope of the constraint.

Variable Selection

- First-Fail (FF) Principle based on estimated filtering *impact*

Impact-Based Search (IBS)

- Idea: Take a branching decision that has the lowest estimated impact in terms of filtering.
- Preliminaries:

$$P = \langle X, D, C \rangle$$

CSP

$$S(P) = \prod_{x \in X} |D(x)|$$

estimate of the size of the search tree

Philippe Refalo.
Impact-based search strategies for constraint programming.
International Conference on Principles and Practice of Constraint Programming (*CP*), 2004.

Impact-Based Search (IBS)

0 $P_0 = \langle X, D, C \rangle$

c_1

1 $P_1 = \langle X, D, C \cup \{c_1\} \rangle$

$$C_{k-1} = C \cup \{c_1, \ldots, c_{k-1}\}$$

k–1 $P_{k-1} = \langle X, D_{k-1}, C_{k-1} \rangle$

$c_k : x = a$

k $P_k = \langle X, D_k, C_k \rangle$

= 0 (no contraction)
= 1 (failure, full contraction)

Contraction of search space with respect to parent node:

$$I^k(x = a) = 1 - \frac{S(P_k)}{S(P_{k-1})}$$

Impact-Based Search (IBS)

- We can estimate over a set K of search nodes the average impact of x=a:

$$\bar{I}(x = a) = \frac{1}{|\mathcal{K}|} \cdot \sum_{k \in \mathcal{K}} I^k(x = a)$$

- This estimate can also be updated at every node, with a forget factor, instead of being averaged:

$$\bar{I}_k(x = a) = (1 - \alpha) \cdot \bar{I}_{k-1}(x = a) + \alpha \cdot I^k(x = a)$$

Exponential moving average $\alpha \in [0,1]$: the higher α is, the faster it will discount older observations.

Impact-Based Search (IBS)

- Estimation of the size of the search tree when trying x=a:

$$S(P) \cdot (1 - \bar{I}(x = a))$$

= 0 (no contraction)
= 1 (failure, full contraction)

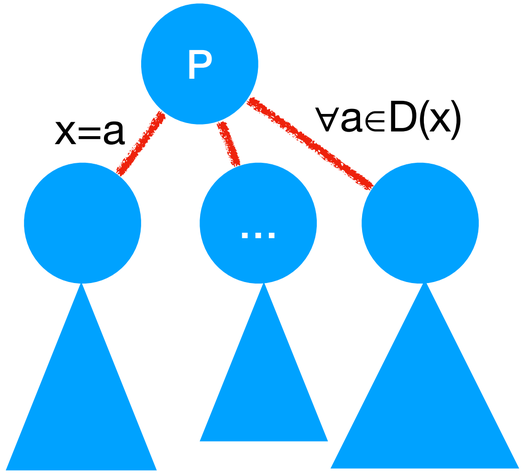
- Estimation of the size of the search tree if labeling the variable x:

$$\sum_{a \in D(x)} S(P) \cdot (1 - \bar{I}(x = a))$$

constant whatever the variable we branch on

- Impact of a variable:

$$I(x) = \sum_{a \in D(x)} (1 - \bar{I}(x = a))$$



Variable heuristic:
Select *x* with minimum $I(x)$
Value heuristic:
Select *a* with minimum $\bar{I}(x = a)$

Variable Selection

- First-Fail (FF) Principle based on estimated filtering *activity*

Activity-Based Search (ABS)

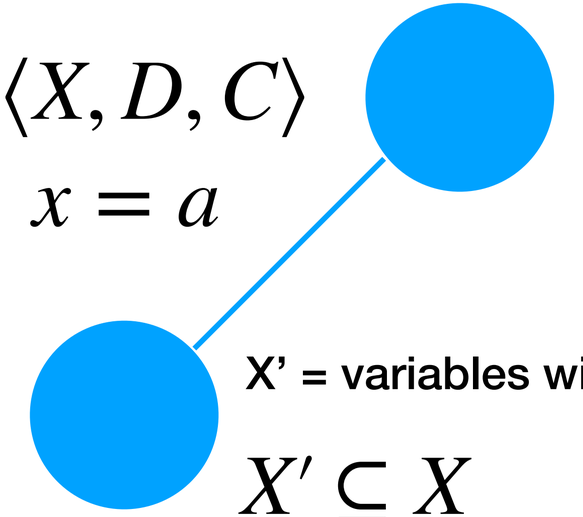
- Idea: Track how often a variable domain is contracted during search, as this allows some learning.
- We expect that a variable whose domain is often contracted is a good candidate to branch on early in order to reduce the depth of the search tree.

Activity-Based Search (ABS)

Each time a search choice is executed, increment the counter A(x), denoting the activity of x, of every variable x having a domain contraction:

$$P = \langle X, D, C \rangle$$

$$x = a$$



Laurent Michel and Pascal Van Hentenryck. Activity-based search for black-box constraint programming solvers. *CPAIOR*, 2012.

X' = variables with domain changed

$$X' \subseteq X$$

decay factor in [0..1]

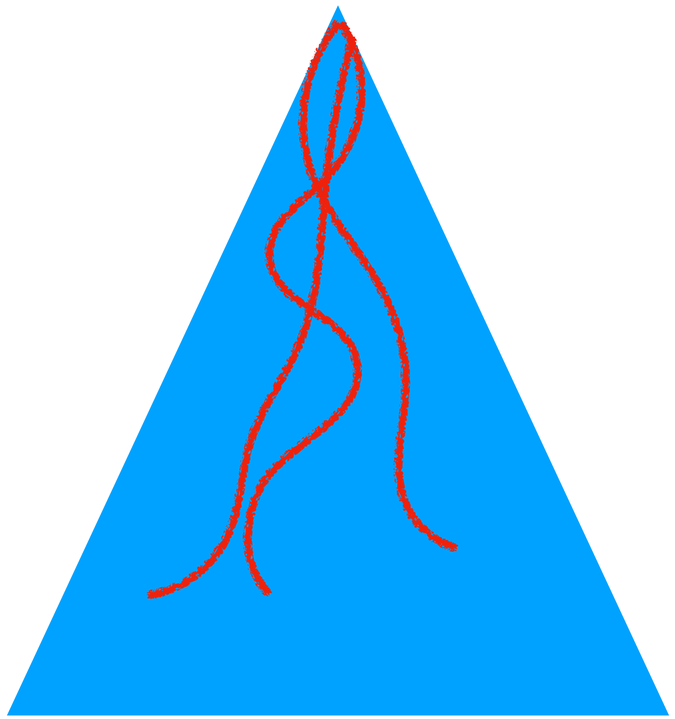
$$\forall x \in X \; where \; |D(x)| > 1 : A(x) := A(x) \cdot \gamma$$

$$\forall x \in X' : A(x) := A(x) + 1$$

Variable Heuristic:
Select *x* maximizing $\dfrac{A(x)}{|D(x)|}$

Initialization of Learning Heuristics

Activity-based search and impact-based search are initialized with some random dives before starting a complete search:



Variable Selection

- First-Fail (FF) Principle based on most recent conflicts

Last-Conflict Search

- Can be used in combination with another heuristic; let us call it fallBackHeuristic.

- lastConflictVariable stores a reference, initialized to null, to a decision variable: this variable is the last one we branched on that led to failure of the fixpoint algorithm.

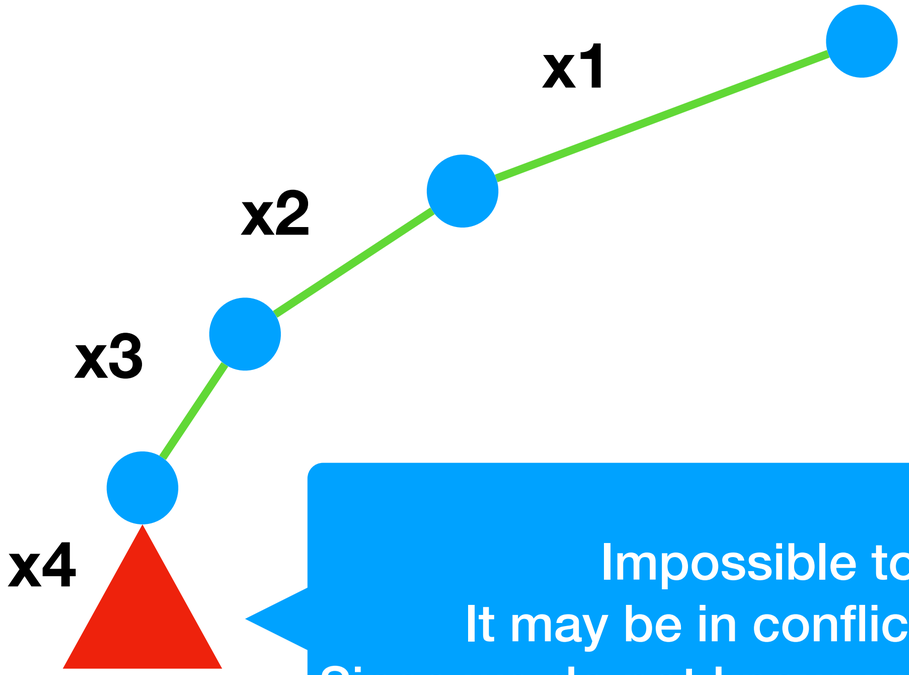
```
nextVarToBranchOn(X) {
    if (lastConflictVariable == null) {
        return fallBackHeuristic(X)
    } else {
        return lastConflictVariable
    }
}
```

This variable caused the most recent conflict, so we may legitimately believe that branching on it may cause a conflict again (since not that much has changed).

Christophe Lecoutre, Lakhdar Saïs, Sébastien Tabary, and Vincent Vidal. Reasoning from last conflict(s) in constraint programming. *Artificial Intelligence*, 2009.

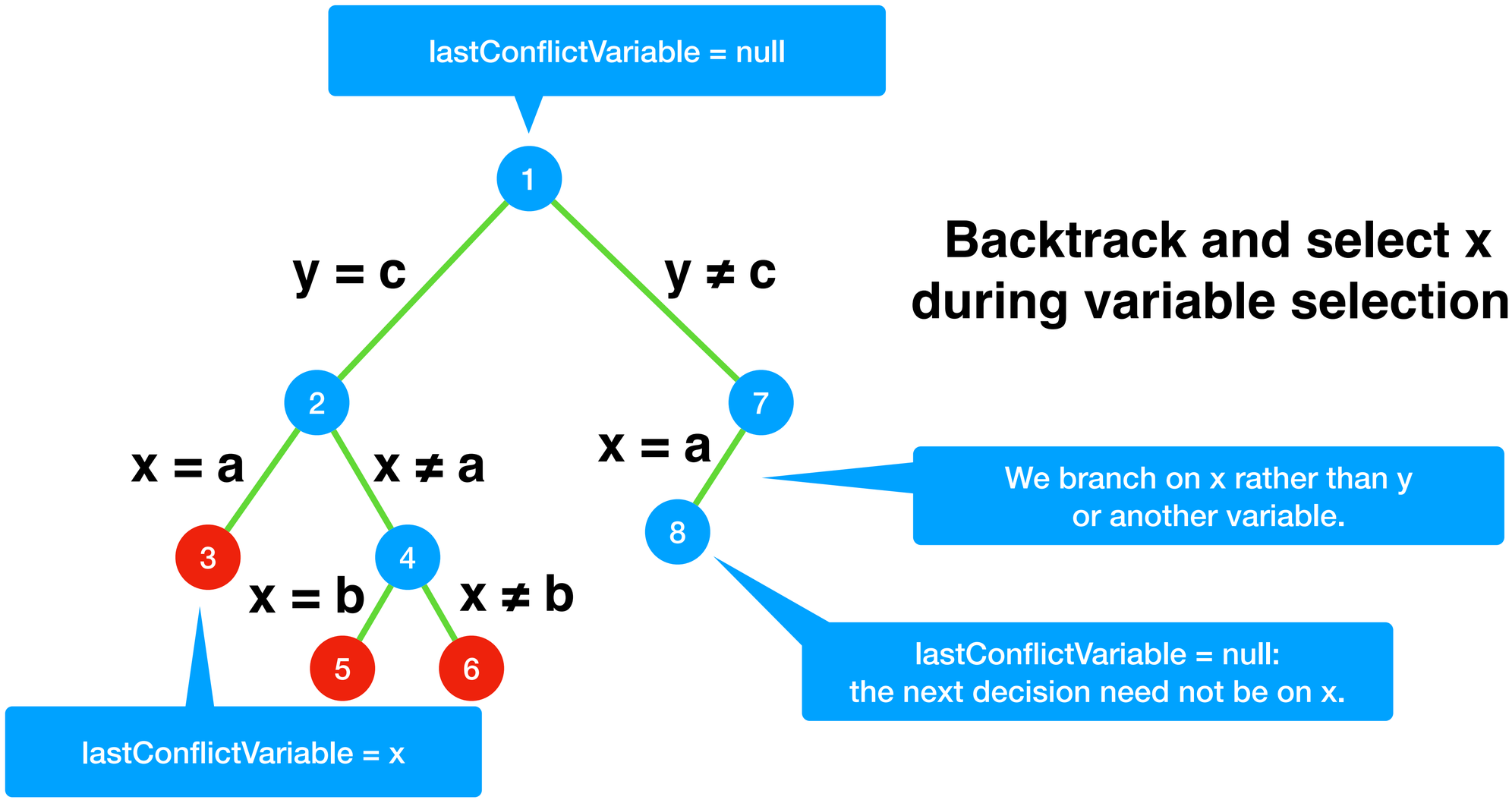
Last-Conflict Search

When branching on a variable leads to a failure,
always branch on this variable first until it is successfully fixed.



Impossible to fix x4 to some value. Why?
It may be in conflict with the choices on x3, x2, or x1.
Since we do not know, we should always try to branch on x4 first,
also after backtracking, as long as this conflict persists.

Last-Conflict Search: Example



Backtrack and select x during variable selection.

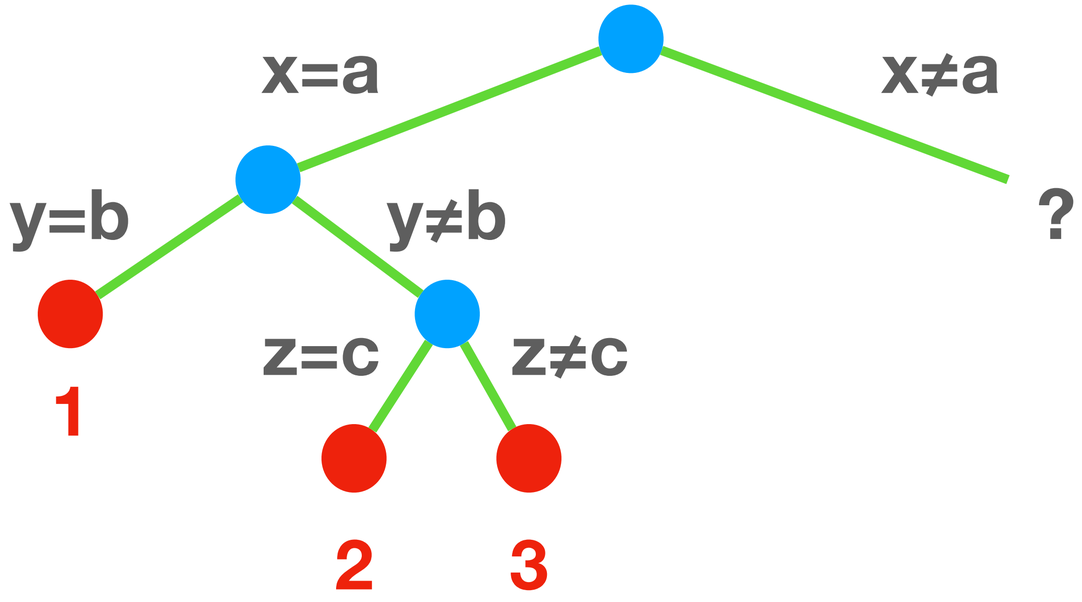
Conflict-Ordering Search (COS)

- Generalization and extension of last-conflict search:
  - Each branching decision is timestamped with a shared counter, which is incremented at each failed visited node of the search tree.
  - Each variable has a timestamp: it is the timestamp of the most recent node where a failure occurred when branching on this variable, and null if it caused no failure so far.
  - COS: Select the unfixed variable with the largest timestamp. If no unfixed variable has a failure timestamp yet, then use a fallback heuristic.

Steven Gay, Renaud Hartert, Christophe Lecoutre, and Pierre Schaus.
Conflict ordering search for scheduling problems.
International Conference on Principles and Practice of Constraint Programming, 2015.

Conflict-Ordering Search (COS)

- A timestamp is associated with each variable and is increased (to the node timestamp) at each failure it causes.
- COS: Select the variable with the largest timestamp.



Assume y≠b leads to fixing y

Variable	x	y	z
Time-stamp	null	1	3

Conflict-Ordering Search (COS)

Algorithm 1: $COS(P = (\mathcal{X}, \mathcal{C})$: CSP$)$

Output: true iff P is satisfiable

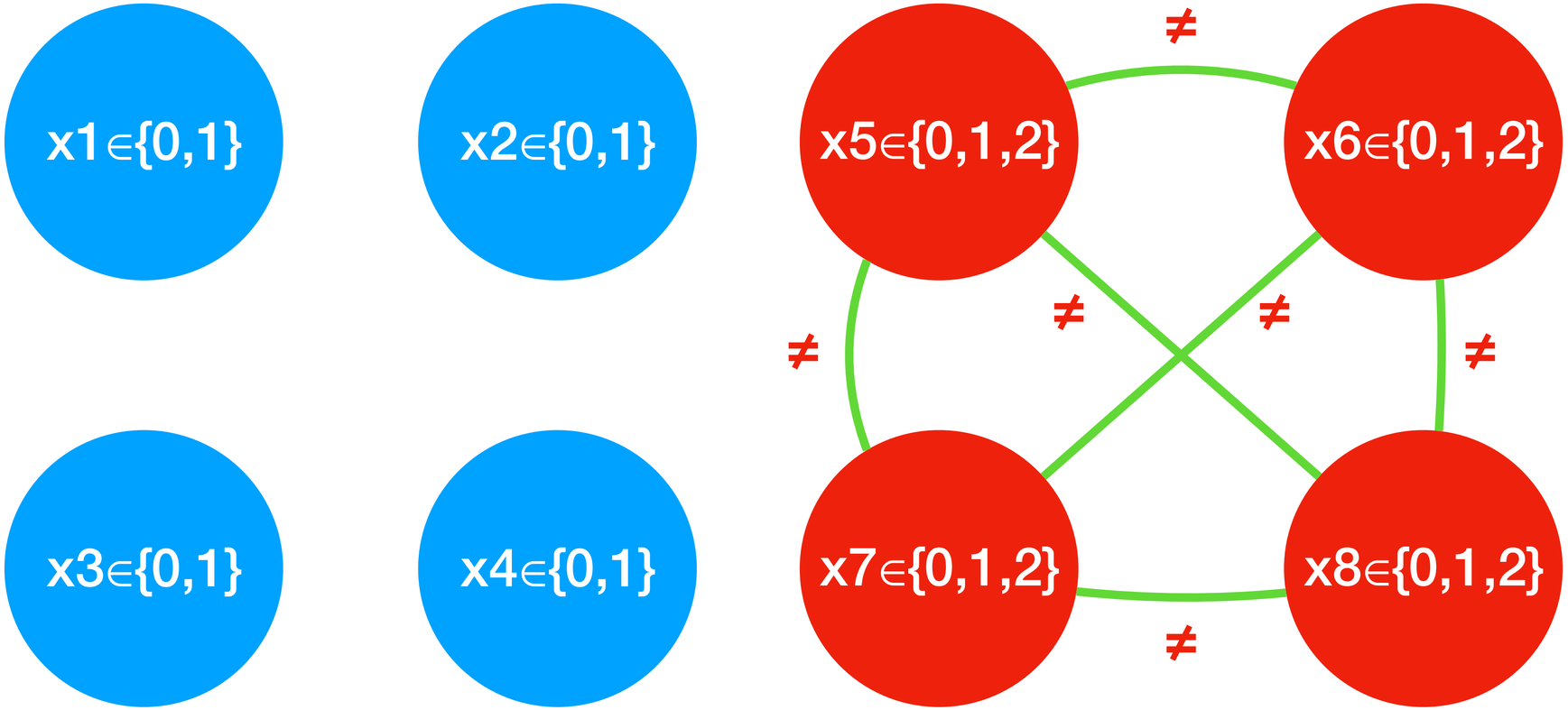
```
P ← φ(P)
if P = ⊥ then
    if lastVar ≠ null then
        nConflicts ← nConflicts + 1
        stamp[lastVar] ← nConflicts
    return false
if ∀x ∈ 𝒳, |dom(x)| = 1 then
    return true
failed ← {x ∈ 𝒳 : stamp[x] > 0 ∧ |dom(x)| > 1}
if failed = ∅ then
    lastVar ← varHeuristic.select()
else
    lastVar ← argmax_{x∈failed}{stamp[x]}
v ← valHeuristic[lastVar].select()
return COS(P|lastVar≤v) ∨ COS(P|lastVar>v)
```

Failure: update the timestamp.

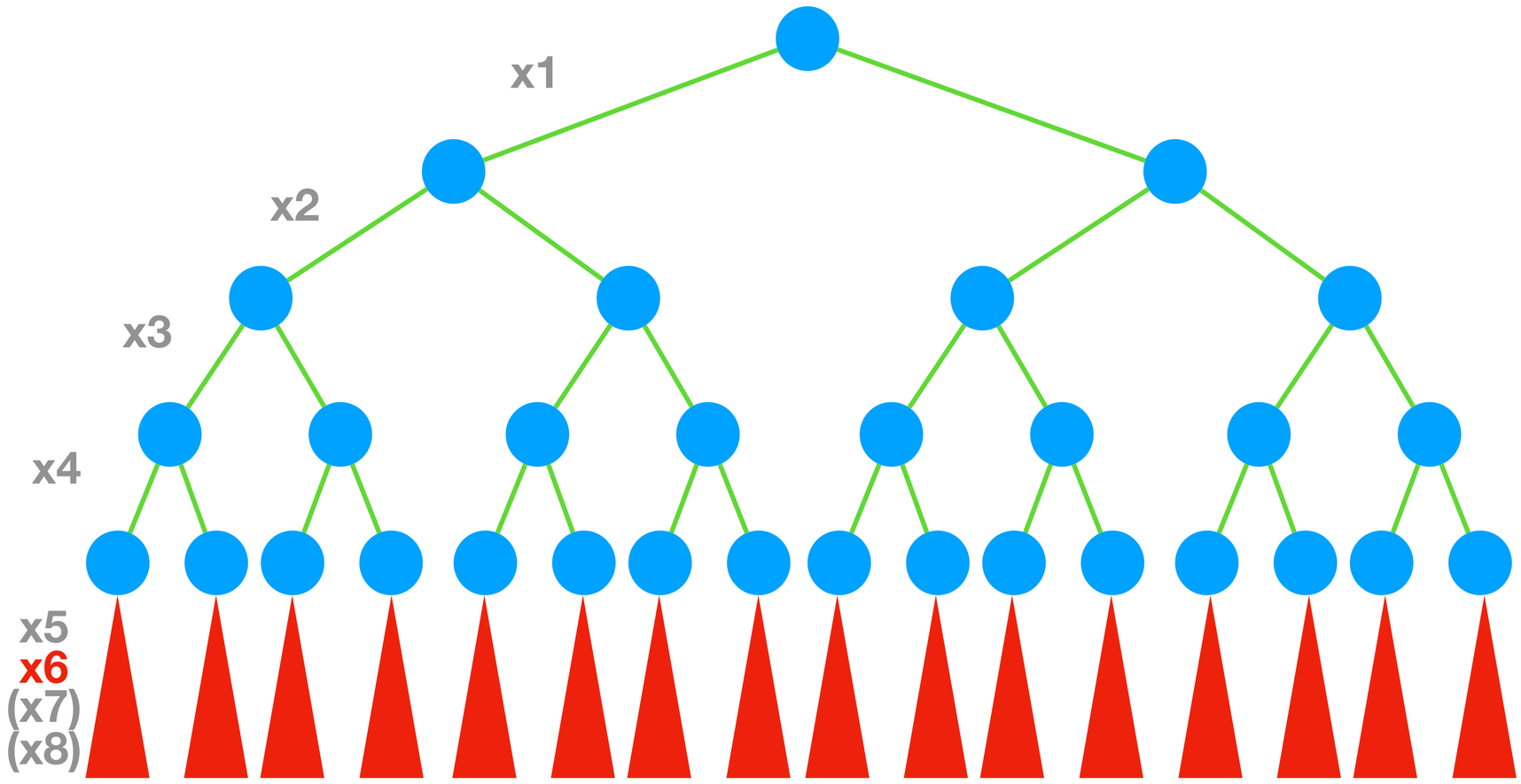
Failure: select the variable with the largest timestamp.

Example

Consider the CSP below, whose infeasibility is not detected by the fixpoint algorithm because of the AllDifferent decomposition. Assume we branch in the order x1, …, x8.

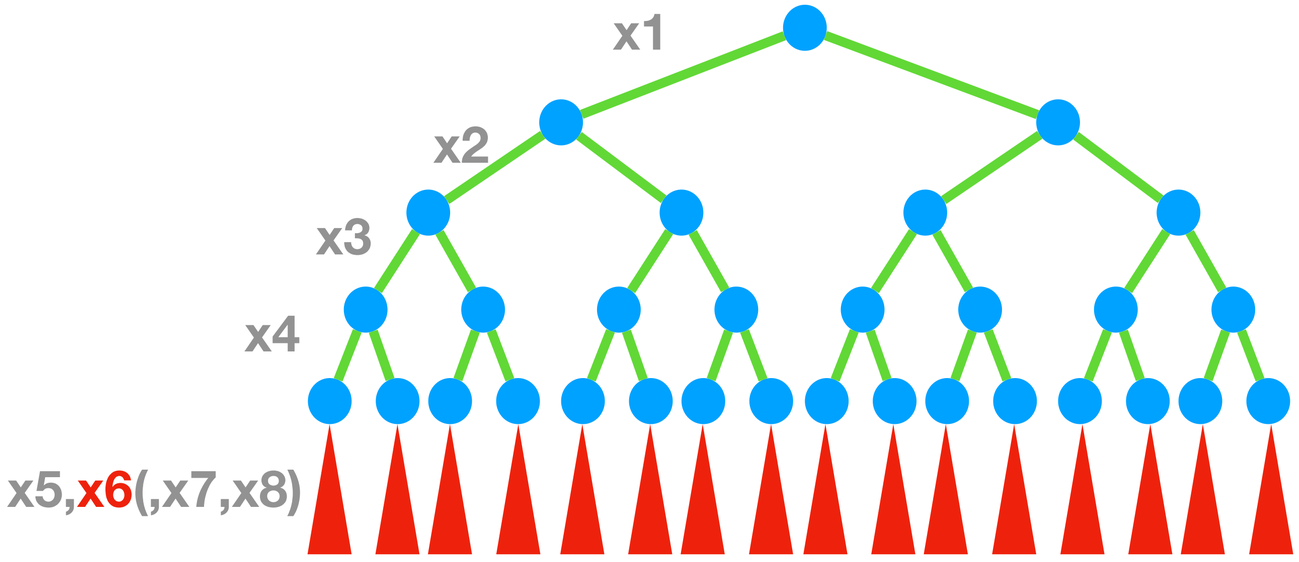


Example: Search Tree



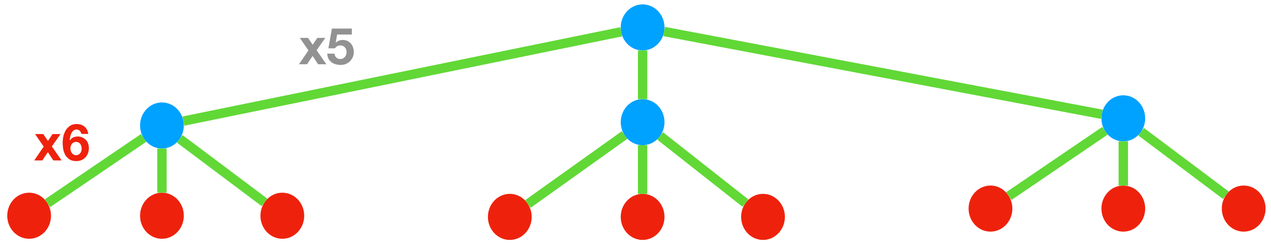
The problem is that there is a subset of variables, {x5,x6,x7,x8}, causing the conflict.
Ideally we should detect this earlier in the search tree.

Example: Search Trees with 2 Static Orders

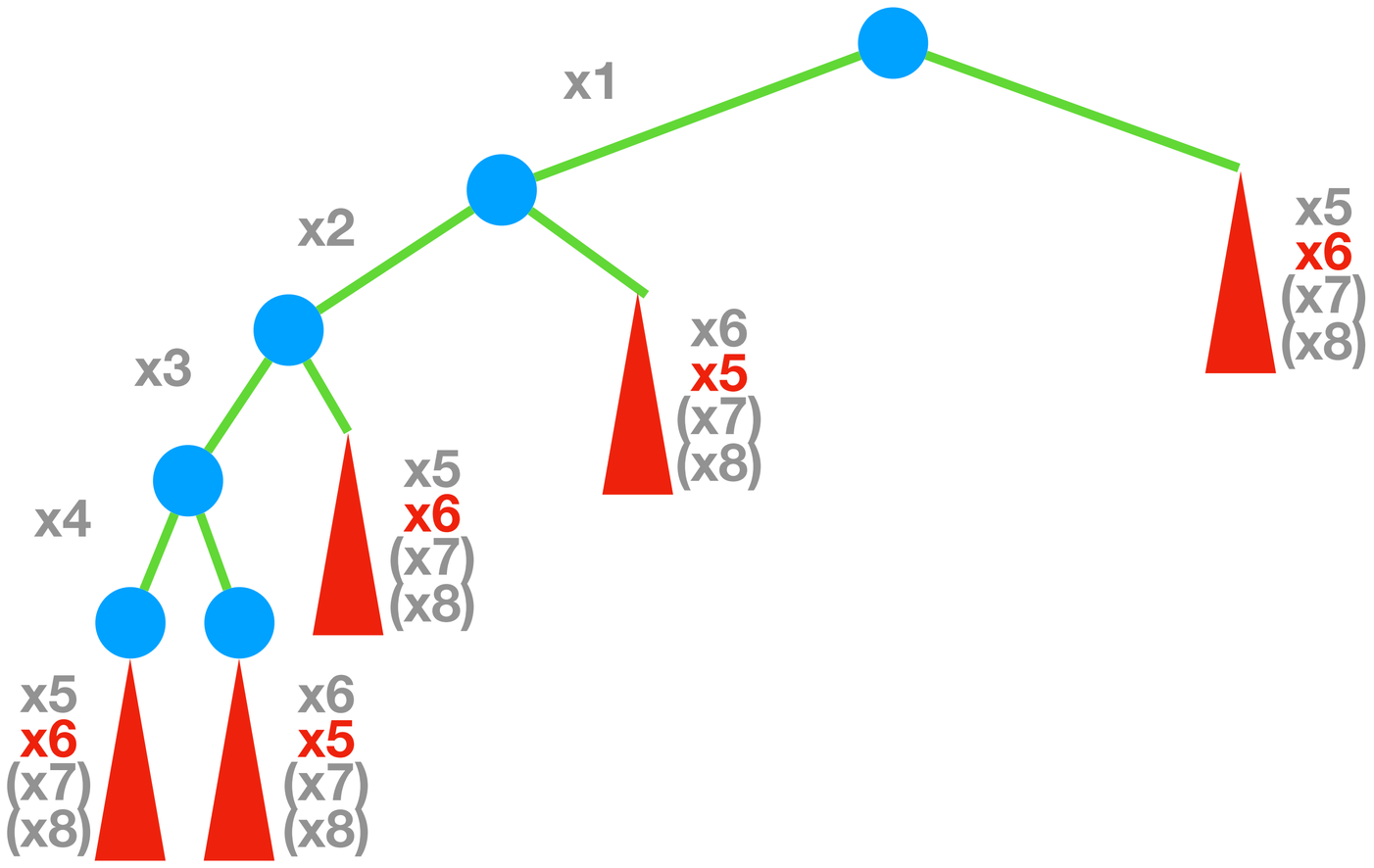


Search tree obtained with the static order [x1,x2,x3,x4,x5,x6,x7,x8].

Search tree obtained with the static order [x5,x6,x7,x8,x1,x2,x3,x4]: ideally, a heuristic learns this order.



Example: Search Tree with COS

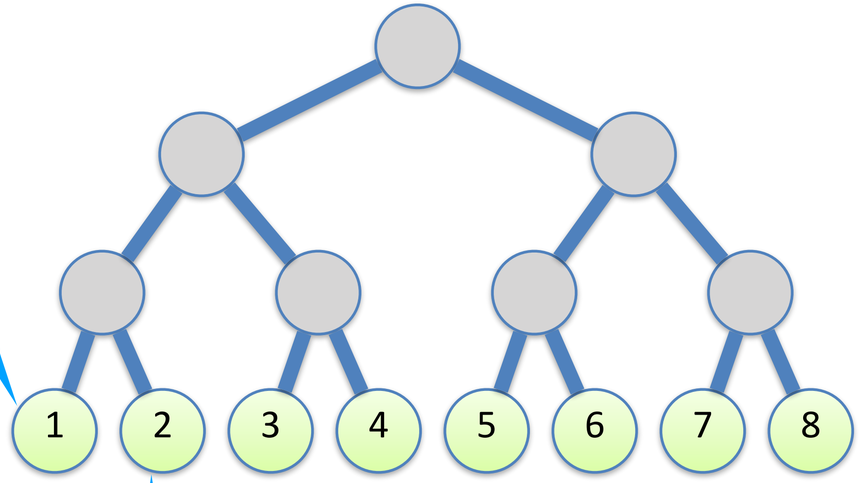


Discrepancy Search

- Very useful if you trust your value heuristic
- Search combinator $\perp$ to the other techniques

Question: Assume I have a good heuristic

Then this solution should be really good.



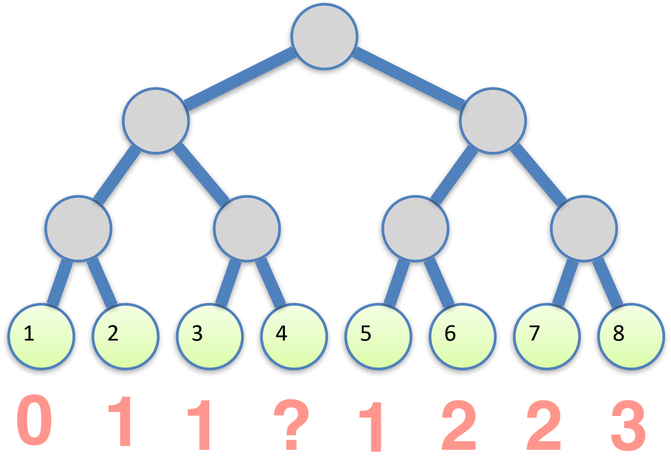
But is this one supposed to be better than…

…this one?

Classify solutions according to the mistakes of the heuristic!

Discrepancy-Based Search

- Discrepancy = the number of right decisions.



Given that:

- I trust my heuristic
- Wrong decisions generally occur at early stages

We should visit solutions by increasing discrepancy.

Combinator: Iterative Discrepancy

```
Objective obj = cp.minimize(totCost);

for (int dL = 0; dL < x.length; dL++) {
    DFSearch dfs = makeDfs(cp, limitedDiscrepancy(firstFail(x),dL));
    dfs.optimize(obj);
}
```

limitedDiscrepancy wraps the search:
it is called a *search combinator*.

Implementation = Exercise

```
public class LimitedDiscrepancyBranching implements Supplier<Procedure[]> {

    private int curD;
    private final int maxD;
    private final Supplier<Procedure[]> bs;

    public LimitedDiscrepancyBranching(Supplier<Procedure[]> branching,
                                       int maxDiscrepancy) {
        if (maxDiscrepancy < 0)
            throw new IllegalArgumentException("max discrepancy should be >= 0");
        this.bs = branching;
        this.maxD = maxDiscrepancy;
    }

    @Override
    public Procedure[] get() {
        // TODO
        throw new NotImplementedException();
    }
}
```

- Eliminate alternatives that would exceed maxD
- Wrap each alternative (closure) such that the call method of the wrapped alternatives:
  - Augments curD depending on its position
  - +0 for alts[0], ..., +i for alts[i]

Restarting

How to take advantage of anything learned…

Restarting Idea

- Two observations
  1. Early mistakes (at shallow depths in the tree) are very costly
     - You need to backtrack all the way up to revise those.
  2. When the variable / value selection **has a learning component**…
  - Its behavior would be different from the start with the learned signal in tow!
- Corollary
  - Give yourself a chance and start *all over again*
  - With the wiser heuristic
  - And a restarting schedule with a limit (e.g. on # of failures) such as Luby

$$S_{F_{lim}} = (1,1,2,\ 1,1,2,4,\ 1,1,2,1,1,2,4,1,1,2,4,8,\ 1,\cdots)$$

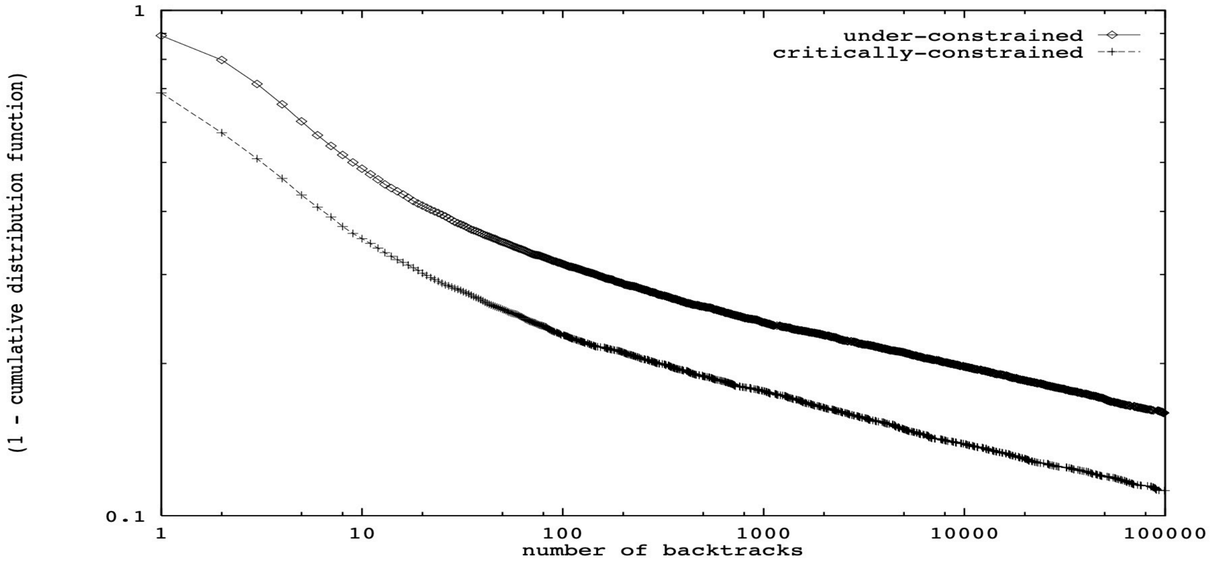
Luby, M., Sinclair, A., Zuckerman, D.: Optimal speedup of Las Vegas algorithms. Inf. Process. Lett. 47(4) (1993) 173–180

Variant

- If
  - You do not have learning
  - But you have randomization in the heuristic(s)
- That works too!

Heavy Tails

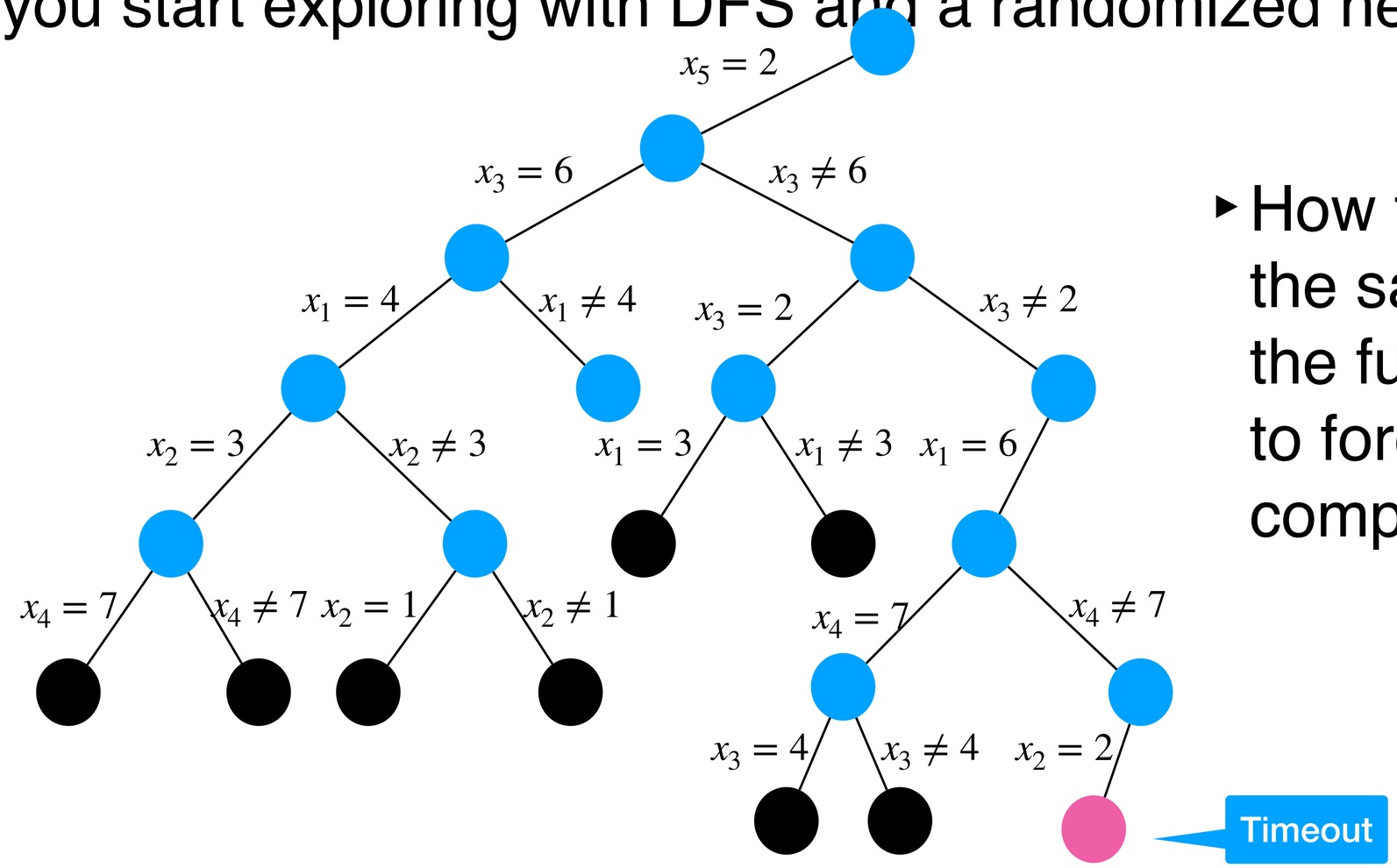
- What is that?
  - Distribution of **runtimes** of a search proce
  - The distribution has “infinite” moments (me
  - Implication
    - Some runs are *very long*
    - Some runs can be quite short
    - Take advantage with restarts when the search is too



Heavy-tailed distributions in combinatorial search
Principles and Practice of Constraint Programming-CP97, 1997, Volume 1330
Carla P. Gomes, Bart Selman, Nuno Crato

Learning from restart, possible ?

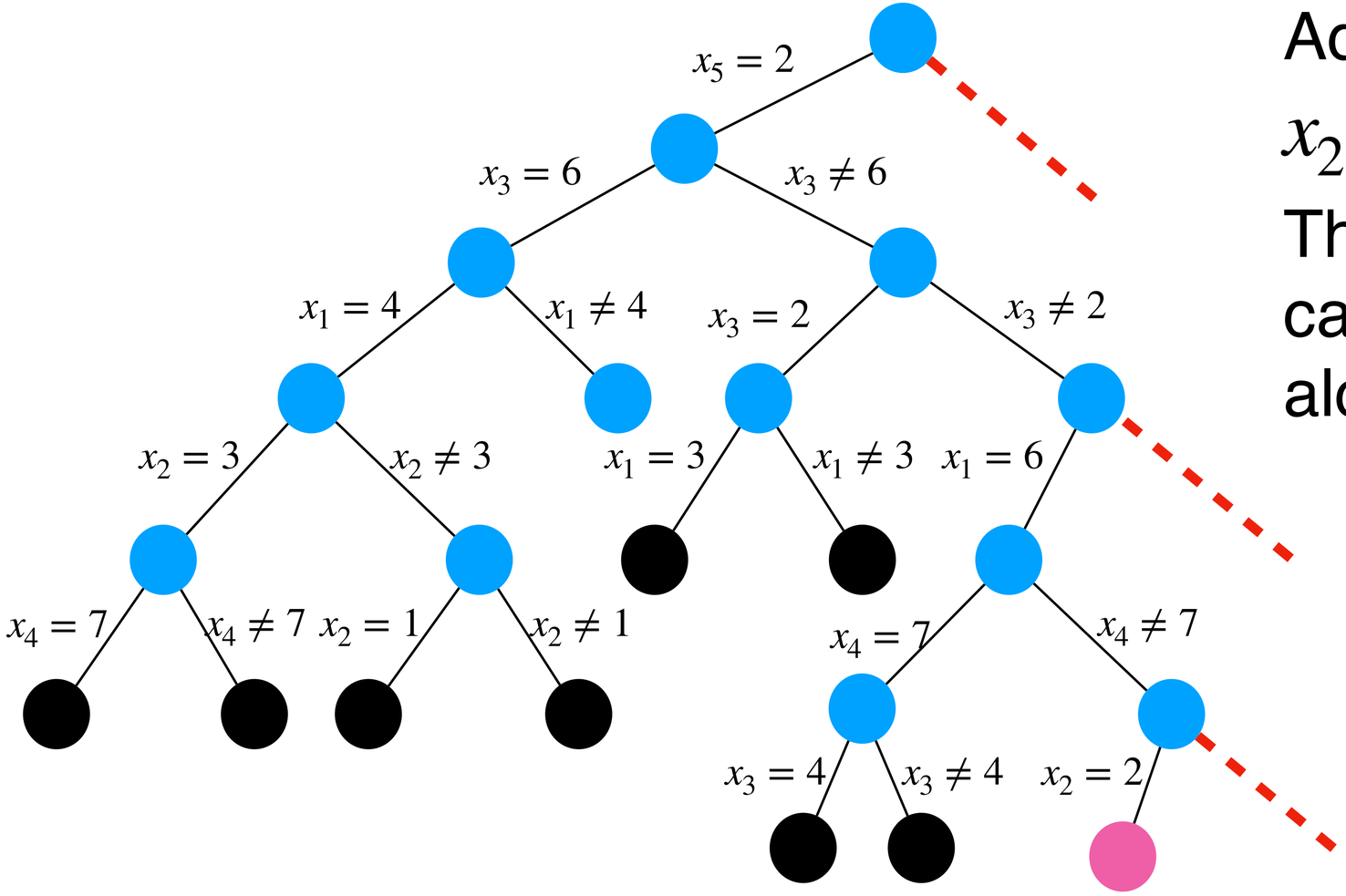
- Imagine that you have set a time-out (or a limit on the number of backtracks) and you start exploring with DFS and a randomized heuristic.



- How to avoid re-exploring the same search space in the future ? Are we doomed to forget the past completely ?

No goods

- Are we doomed to forget the past completely ? NO!



Add the constraint

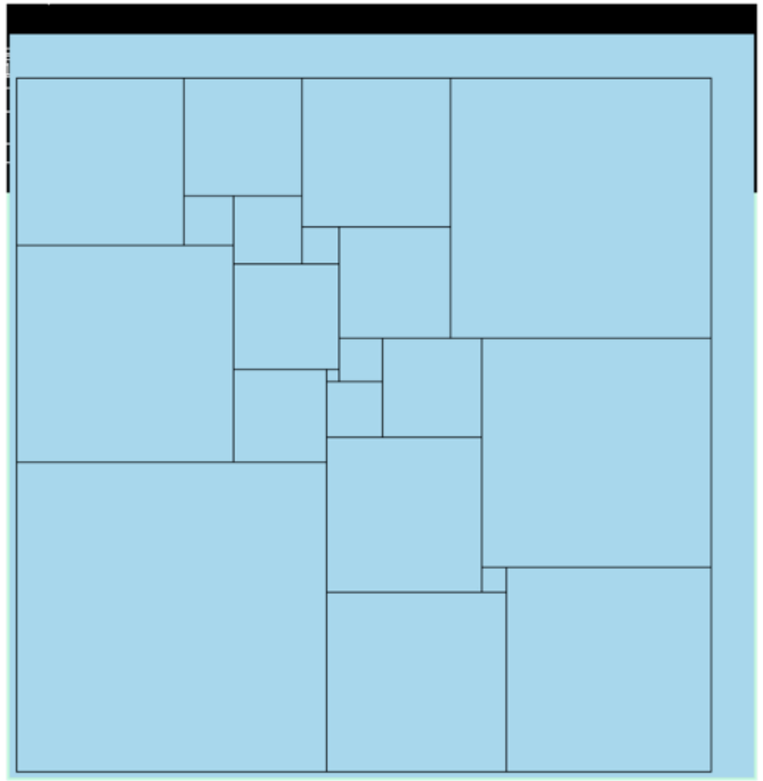
$$x_2 \neq 2 \vee x_1 \neq 6 \vee x_5 \neq 2$$

This is called a No-good. We can accumulate no-goods along the restarts

Perfect Square Packing

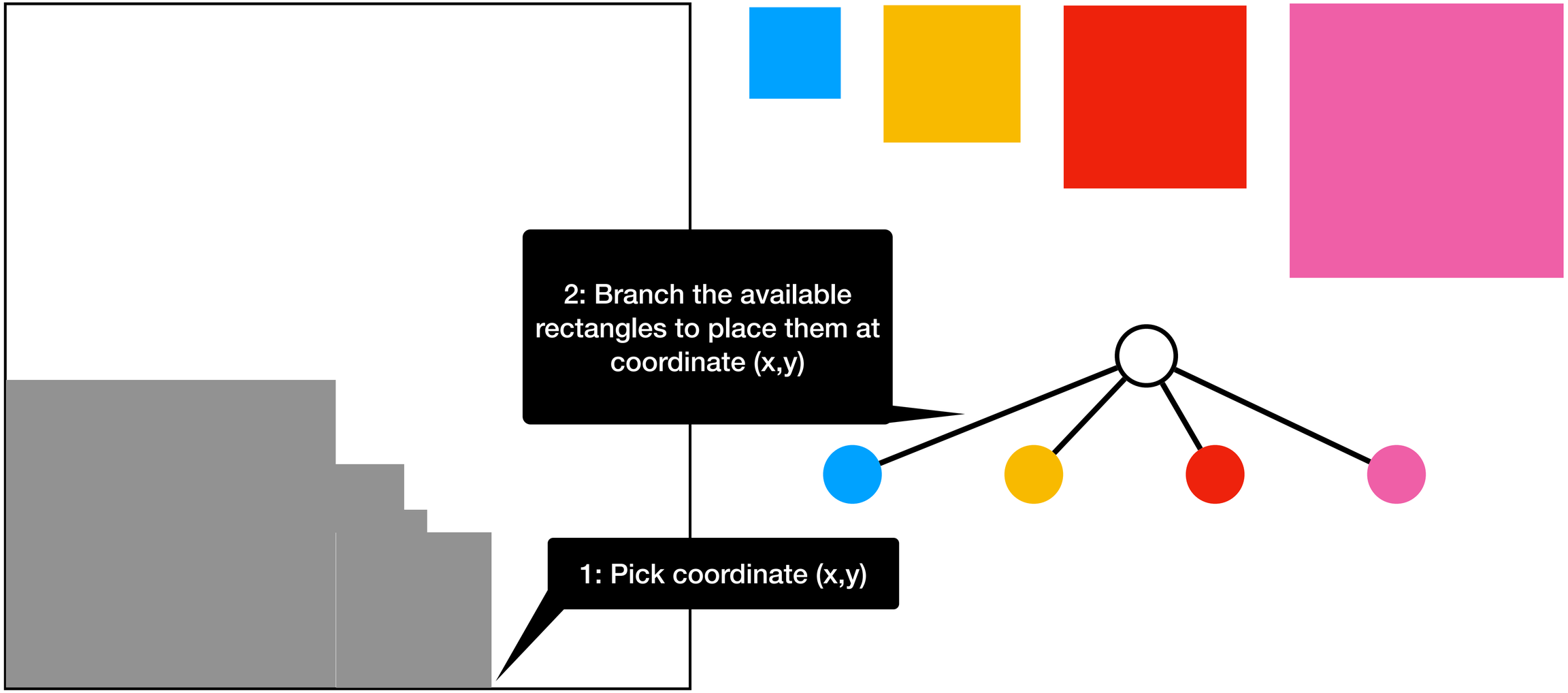
Place squares of different sizes into a big squares such that they do not overlap

```
int s = 112;
int [] side = new int[]{50, 42, 37, 35, 33, 29, 27, 25, 24, 19, 18, 17, 16, 15, 11, 9, 8, 7, 6, 4, 2};
```



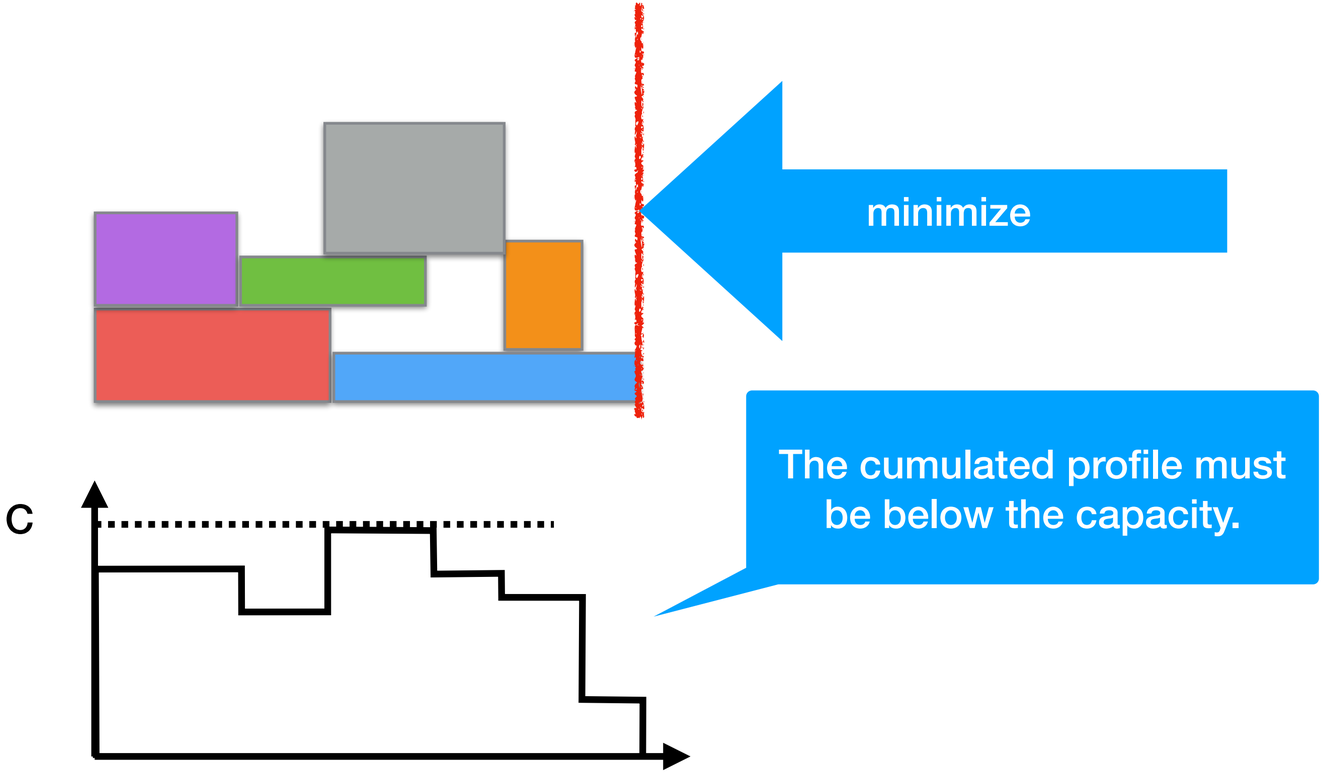
21 rectangles

Perfect Square Packing



Resource Constrained Scheduling Problem

- Color = resource (or: machine), with capacity 1.
- Precedence constraints (denoted ≪) on the activities of a job.



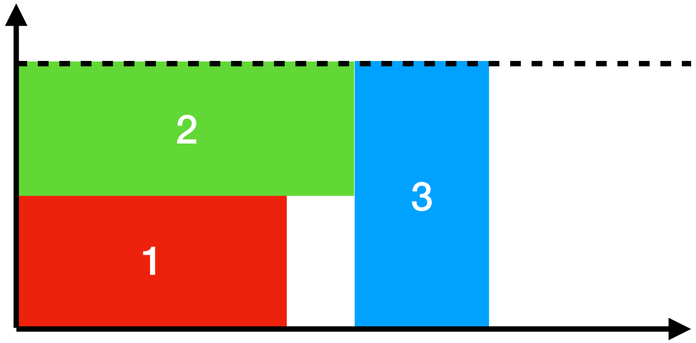
SetTimes search scheme

- Pick the unfixed task i, that is not postponed, with smallest possible start time (est_i)
- On the left branch: Assign the start of the task to est_i
- On the right branch: Set the task *postponed*. It will remain so as long as its est_i does not change
- If we detect that a task would be postponed forever, or if all the unfixed tasks are postponed, then fail and backtrack

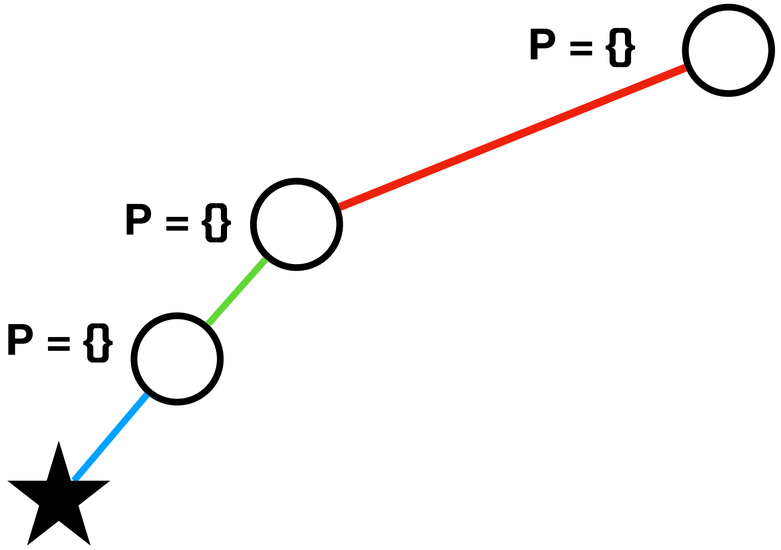
⚠️ complete search for makespan minimization, and simple constraints, like precedences (not if you have start(A)-start(B) <= 3).

Le Pape, C., Couronne, P., Vergamini, D., & Gosselin, V. (1995). Time-versus-capacity compromises in project scheduling.

SetTimes

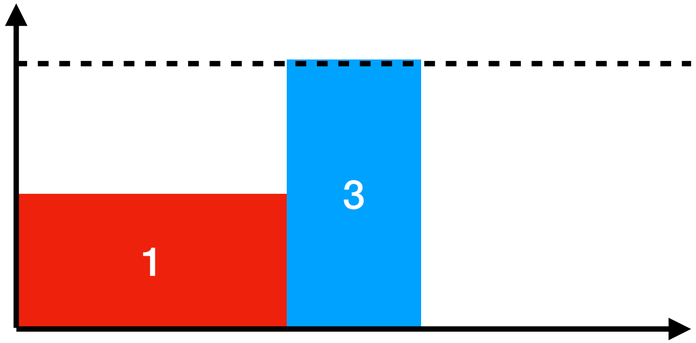


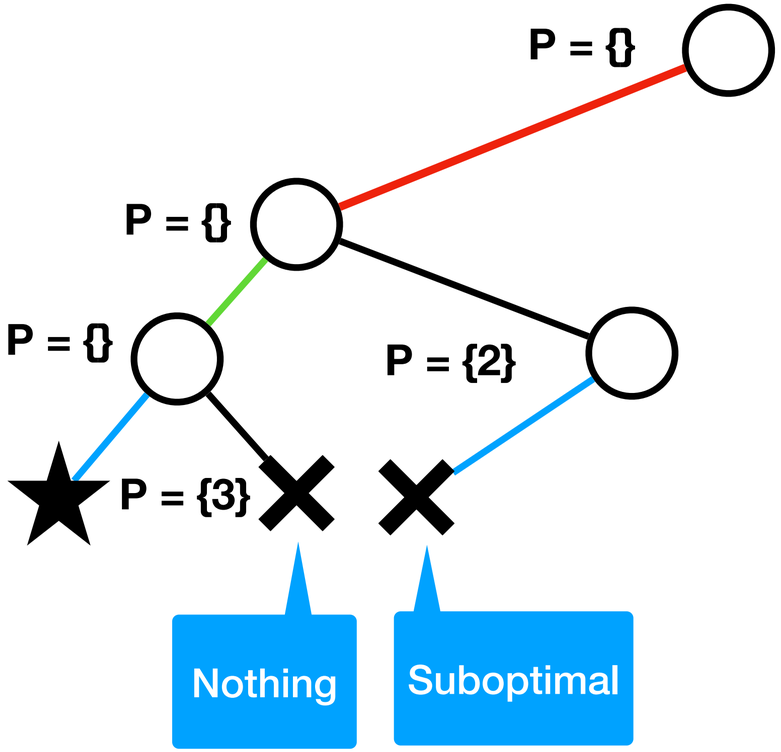
P = set of postponed tasks



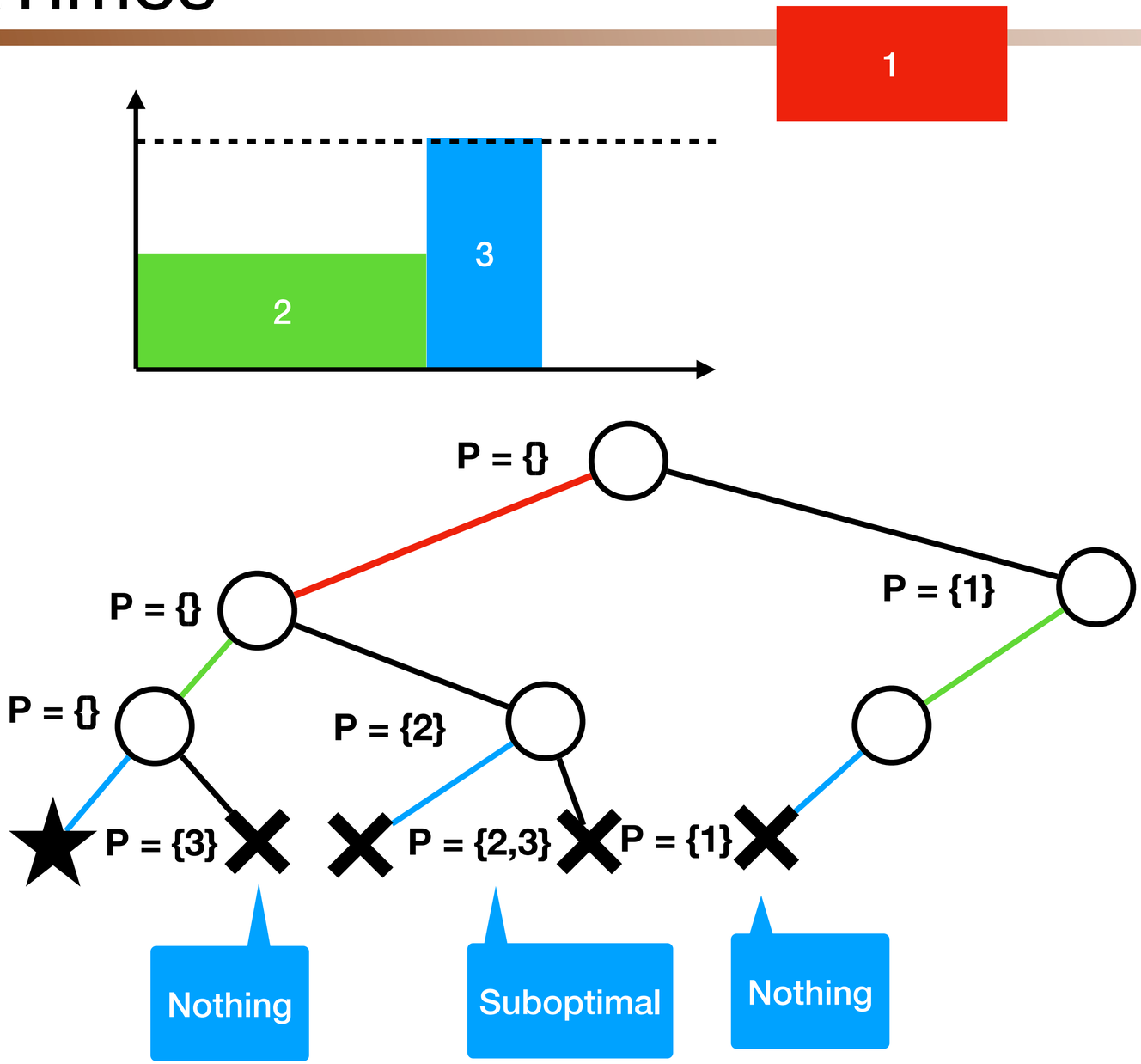
Le Pape, C., Couronne, P., Vergamini, D., & Gosselin, V. (1995). Time-versus-capacity compromises in project scheduling.

SetTimes

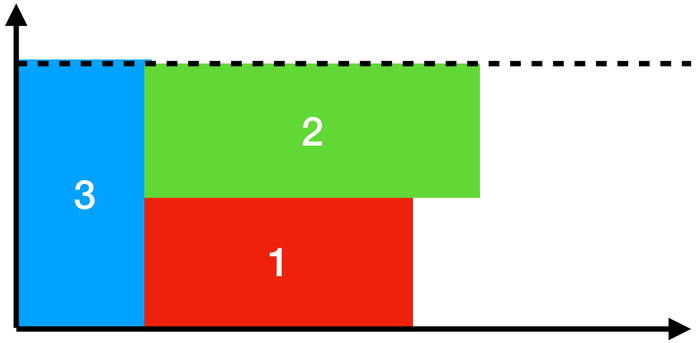


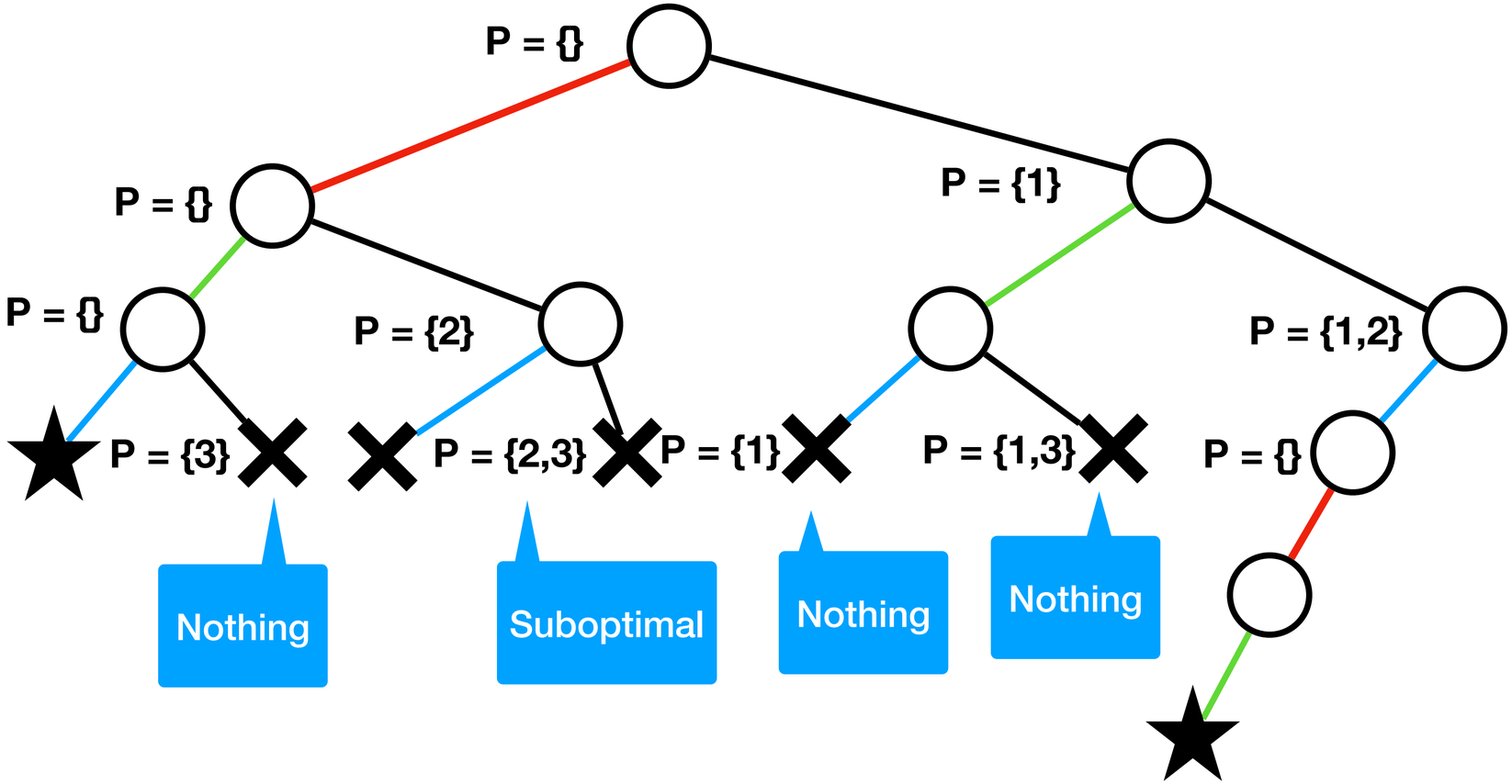


SetTimes

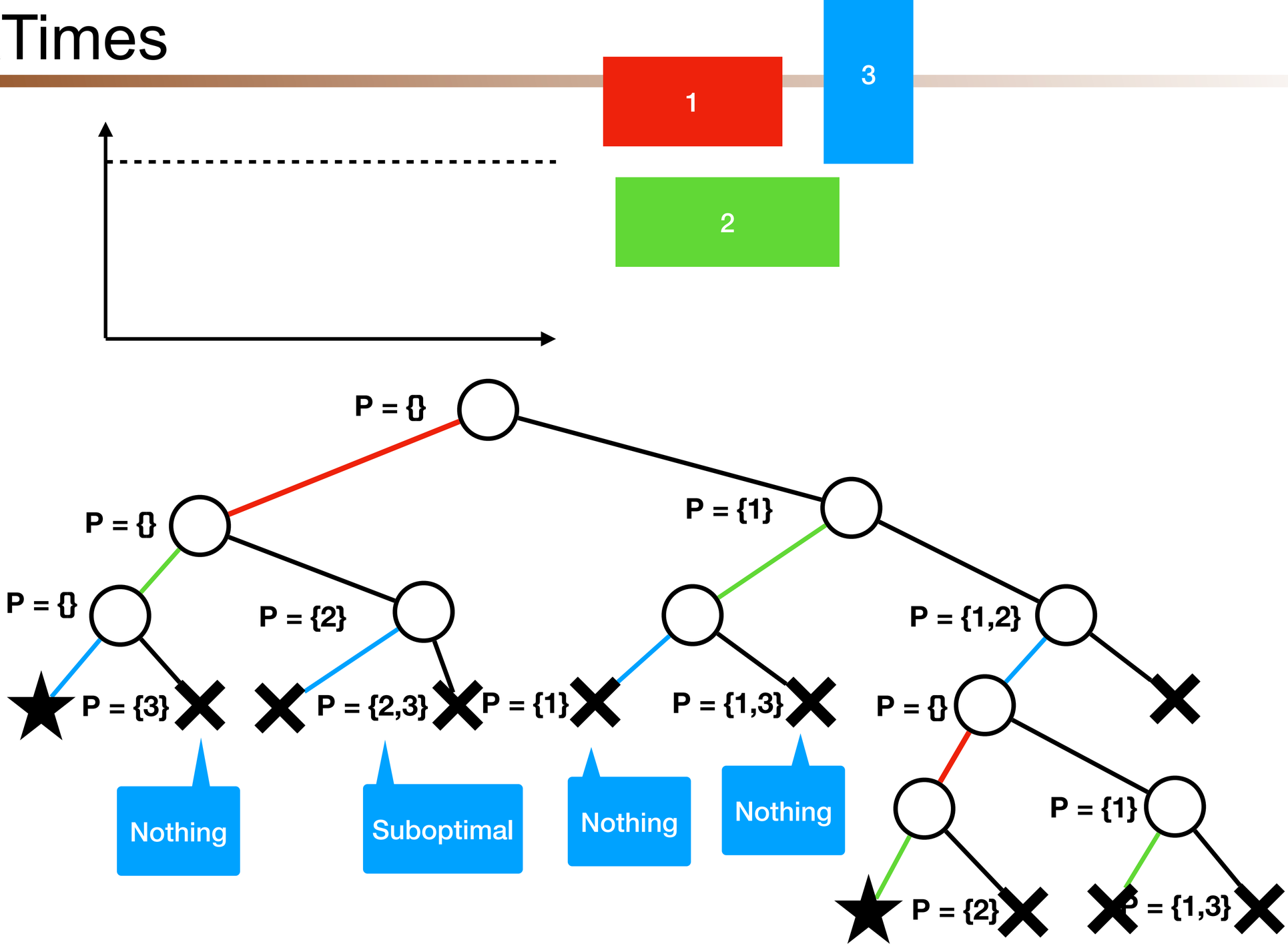


SetTimes


3
2
1

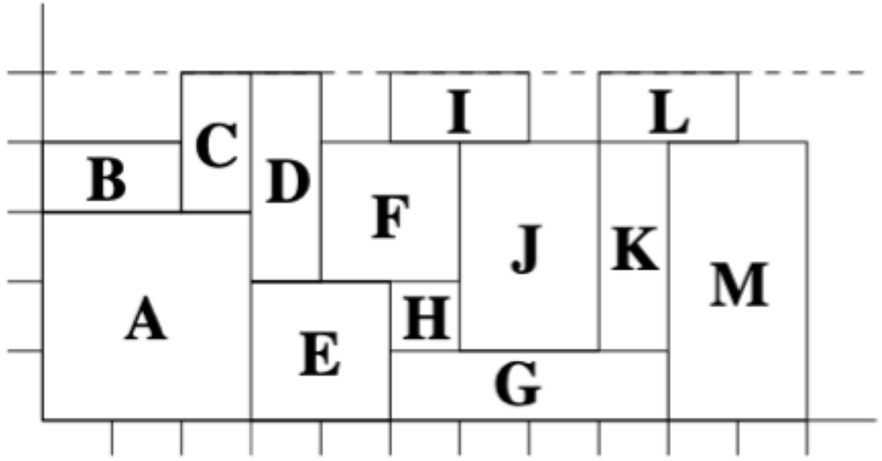

P = {}
P = {}
P = {1}
P = {}
P = {2}
P = {1,2}
P = {3}
P = {2,3}
P = {1}
P = {1,3}
P = {}
Nothing
Suboptimal
Nothing
Nothing

SetTimes


1
3
2
P = {}
P = {}
P = {1}
P = {}
P = {2}
P = {1,2}
P = {3}
P = {2,3}
P = {1}
P = {1,3}
P = {}
P = {1}
P = {2}
P = {1,3}
Nothing
Suboptimal
Nothing
Nothing

LNS for Scheduling

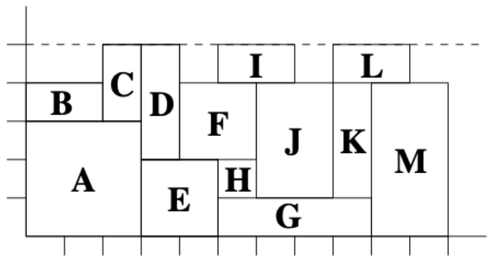
- If you relax some start-times while keeping fixed other start-times, you don't have much chance to improve the makespan at next iteration
- Idea: relax all start times, keep some precedence constraints (keep relative order constraints from the the best solution)



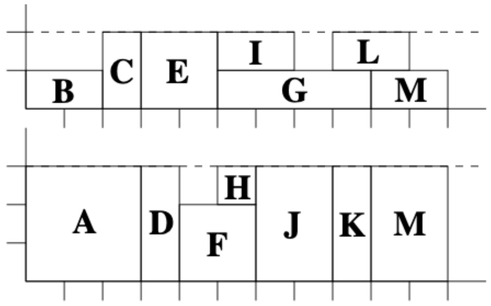
Solution

Godard, D., Laborie, P., & Nuijten, W. Randomized Large Neighborhood Search for Cumulative Scheduling. In *ICAPS* 2005

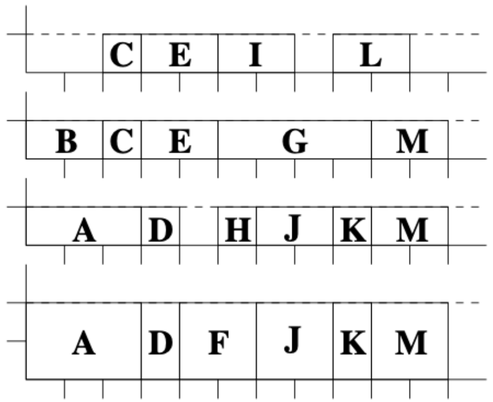
LNS for Scheduling: Partial Order Schedule



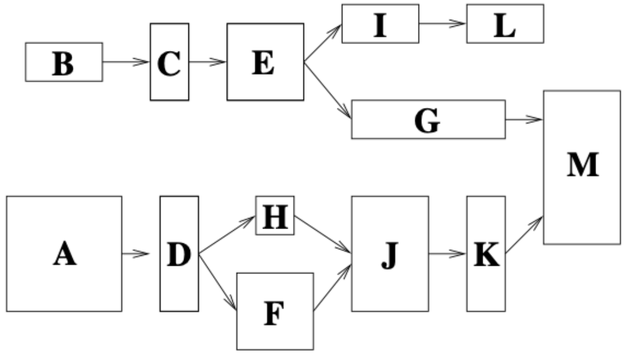
Solution



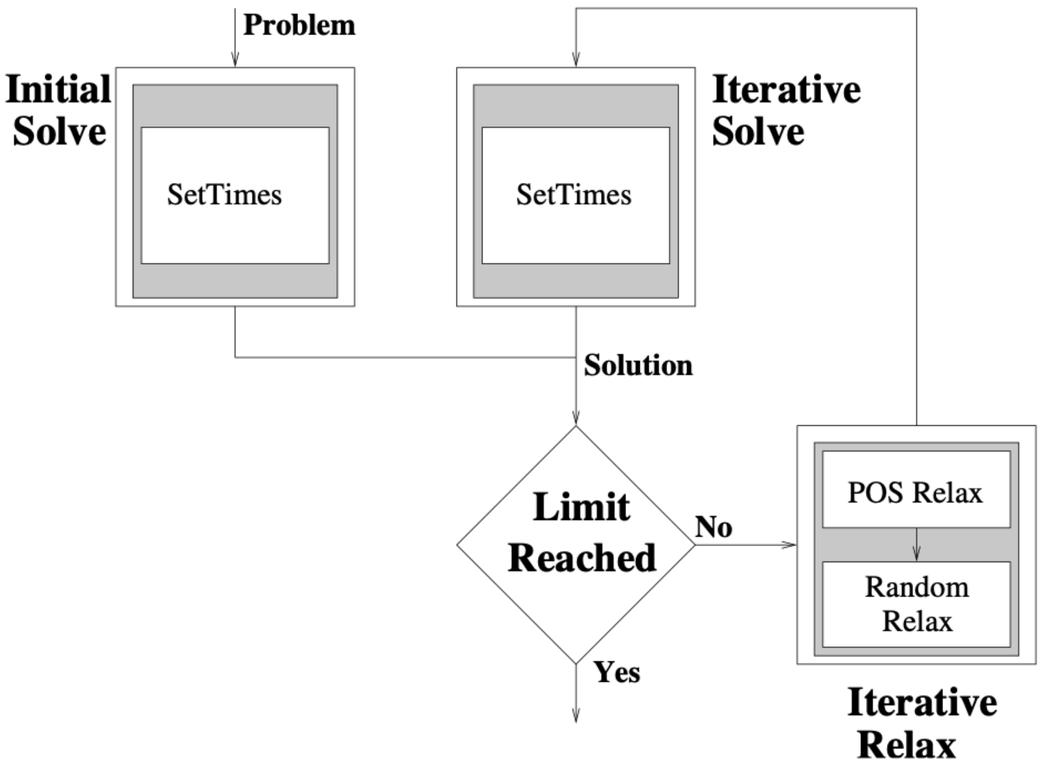
First split of resources



Second split of resource



Final POS of resource



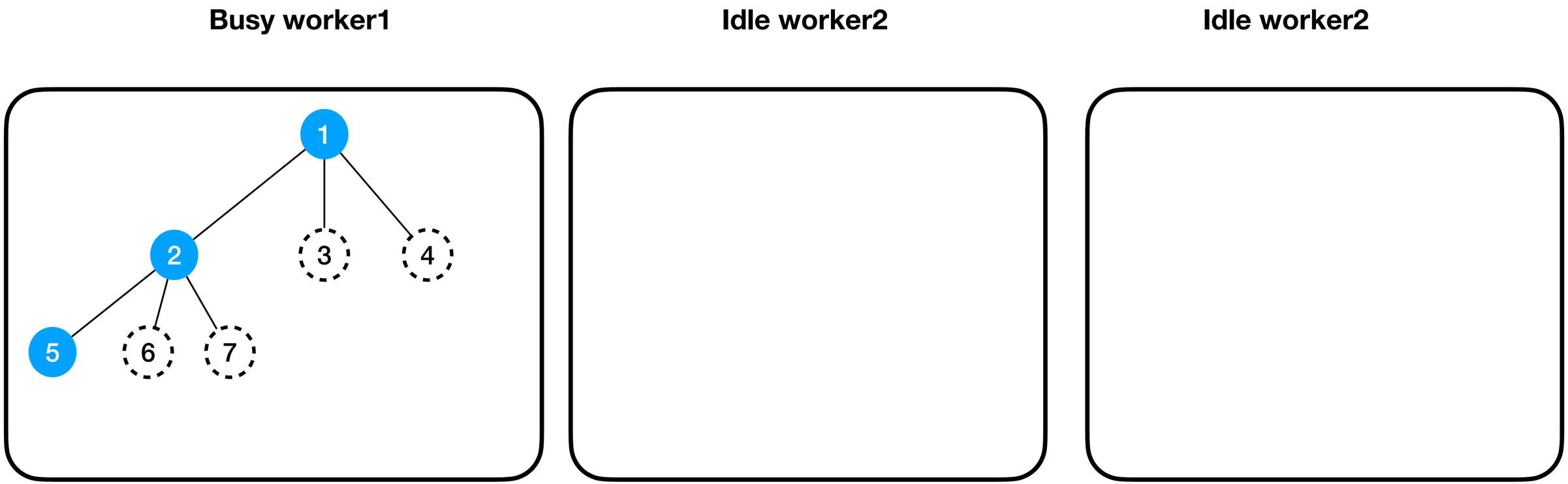
Godard, D., Laborie, P., & Nuijten, W. Randomized Large Neighborhood Search for Cumulative Scheduling. In *ICAPS* 2005

Parallelization in CP

You have more than one processor, what can you do ?

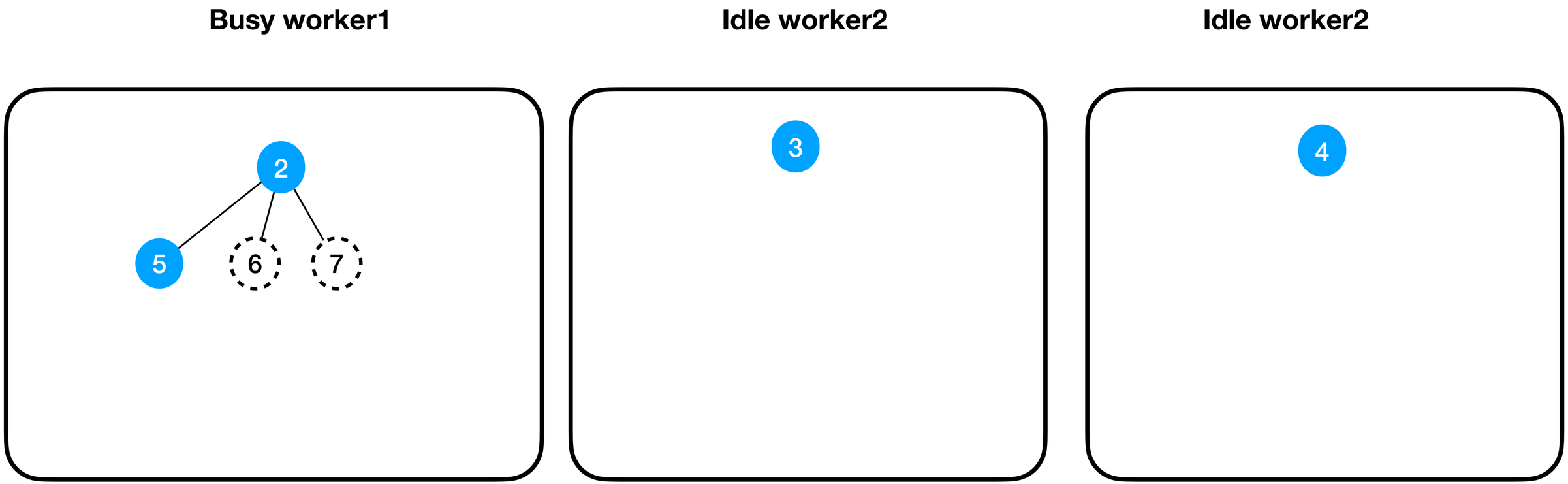
- Portfolio: use several heuristics (or randomized search)
  - For COP: communicate the bounds when a new best solution is found
  - If you use restart, you can share the No-Goods (not always easy)
- If you want to stay complete two strategies:
  - Work-stealing (quite a lot of communication):
    - Search Procedures and Parallelism in Constraint Programming. Perron, L. CP1999
    - Michel, L., See, A., & Van Hentenryck, P. Parallelizing constraint programs transparently. CP2007
    - Chu, G., Schulte, C., & Stuckey, P. J. Confidence-based work stealing in parallel constraint programming. CP2009
  - Embarrassingly parallel search (try to keep communication very low):
    - Régin, J. C., Rezgui, M., & Malapert, A. Embarrassingly parallel search. CP2013

Work Stealing



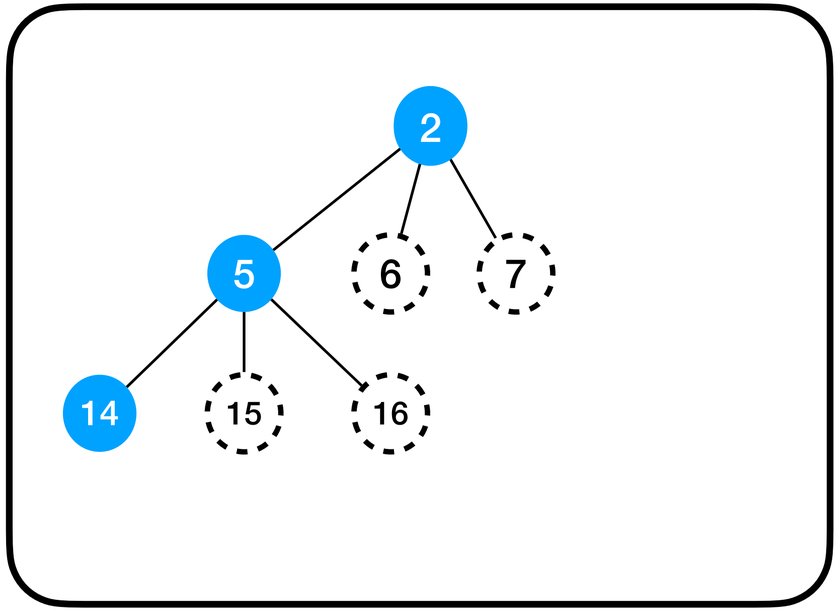
Michel, L., See, A., & Van Hentenryck, P. Parallelizing constraint programs transparently. CP2007

Work Stealing

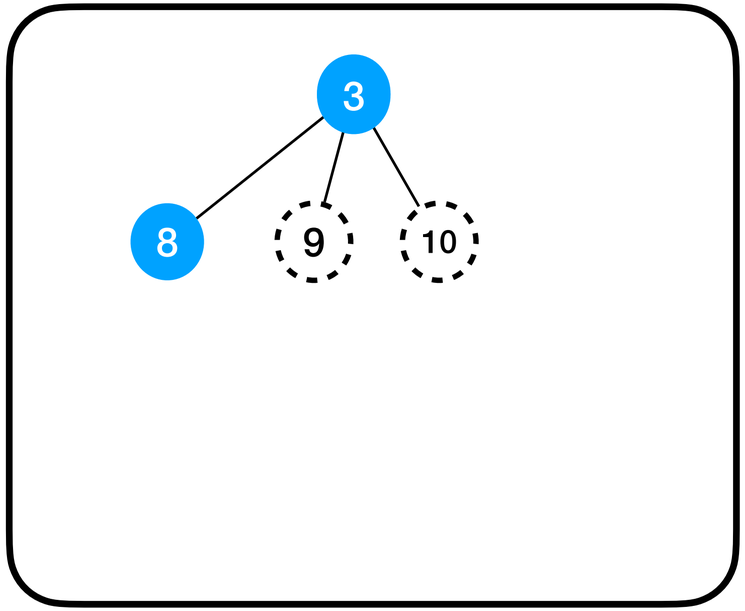


Work Stealing

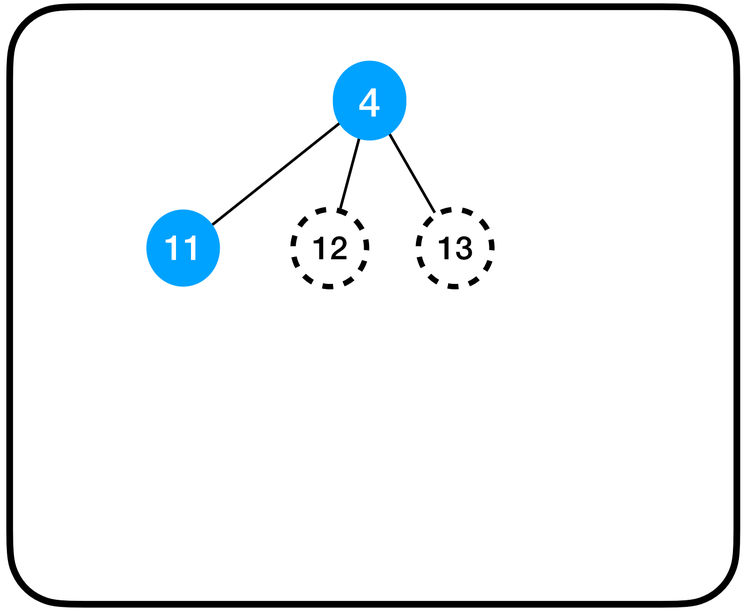
Busy worker1



Idle worker2

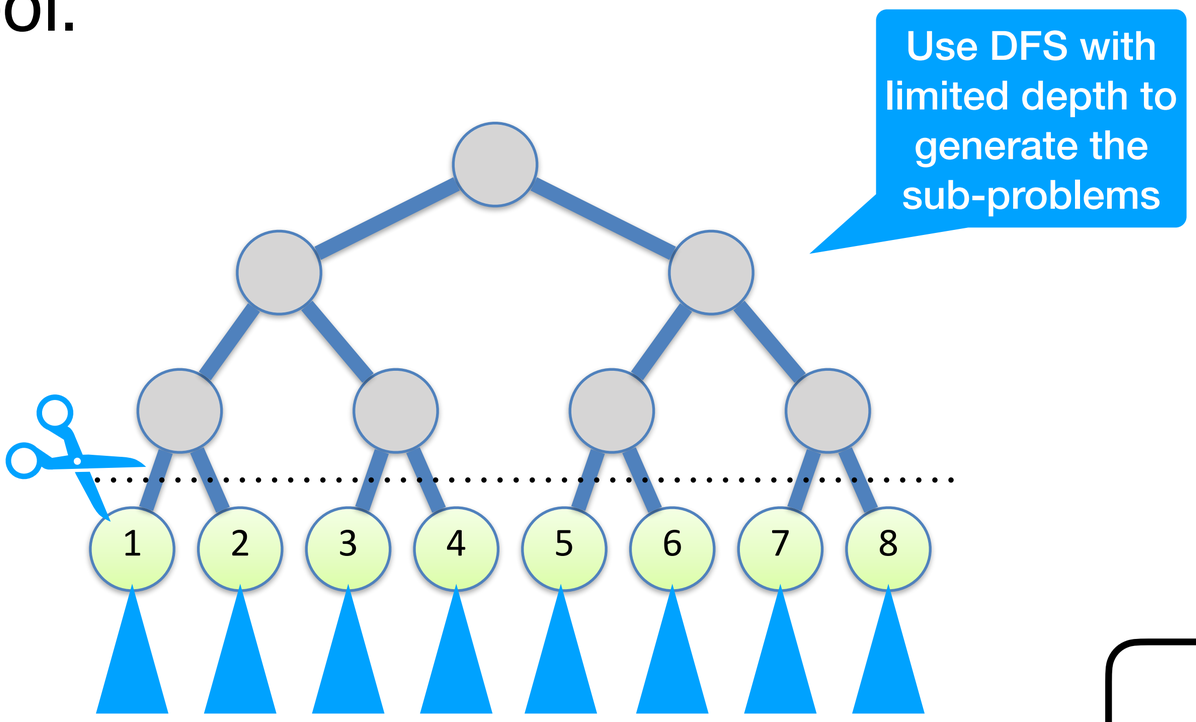


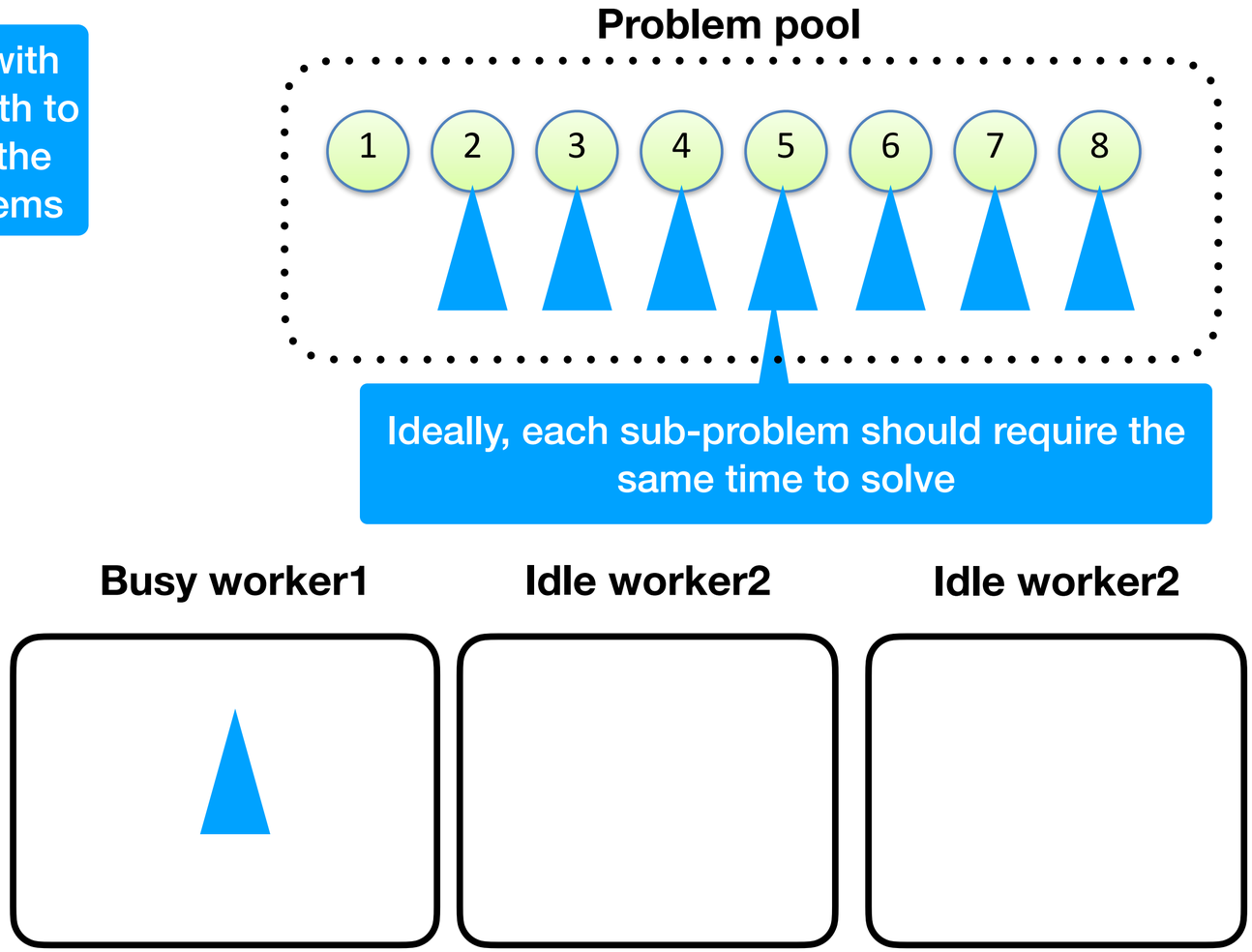
Idle worker2



Embarrassingly parallel search

- Split the initial problem into a huge number of small sub-problems, add them to a pool so that whenever a worker is idle, it can just take the next one in the pool.





- Régin, J. C., Rezgui, M., & Malapert, A. Embarrassingly parallel search. CP2013

Search Wrap up

Search: Summary

- Value selection heuristic:
  - Bound-Impact Value Selector (BIVS)
  - Phase saving
- Variable selection using the first-fail heuristic:
  - Degree and Weighted Degree
  - Impact-Based Search (IBS):
    - Choose the variable and value that have a strong impact in terms of filtering.
    - The impacts are estimated and learned during the search.
  - Activity-Based Search (ABS):
    - Choose the variable whose domain is often filtered: activity.
    - The activity is learned during the search.
  - Last-Conflict Search (LC):
    - Choose the variable that led to the latest conflict, if any.
  - Conflict-Ordering Search (COS):
    - Choose the variable with the largest timestamp.
- Discrepancy-Based Search Combinator